Learn HDFS Architecture

Generated 03/10/2026, 18:35:44 • 48 messages • Source: Shared ChatGPT link
https://chatgpt.com/share/6ac0fd5b-26b8-83ee-a68e-5d187c382033

USER

HDFS Architecture HDFS overview, NameNode, DataNode, Secondary NameNode, blocks, replication, read/write flow, HDFS functions so , i need to learn these things, im person who doesnt know about any of these , so need to explain me from the very start , also inbetween ask me questions to ensure that i get it , sources given are the textbook and the links in the image

ASSISTANT

Yes. We'll learn **HDFS from absolute zero**, and I'll use your **Elmasri textbook as the primary source**, with the Apache Hadoop documentation you were given as the supporting source. The textbook's HDFS section is **25.3**, covering preliminaries, architecture, I/O/replication, and scalability.

I'll also **stop periodically and ask you short questions**. You answer them, I'll correct/clarify if needed, and then we continue. That way we build the mental model instead of memorizing definitions.

1. First: Why do we even need HDFS?

Before learning:

- NameNode
- DataNode
- blocks
- replication
- read/write
- Secondary NameNode

we need to understand **the problem HDFS is solving**.

Imagine a normal computer

Suppose you have a laptop with:

```
Disk
% % %    d a t a . c s v
```

Maybe `data.csv` is 20 GB.

Your operating system knows:

> **"There is a file called `data.csv`, and its contents are stored somewhere on this disk."**

That's ordinary file storage.

But now imagine:

TEXT

```
data.csv = 100 TB
```

You don't have a single computer with a 100-TB disk.

So instead you have many machines:

TEXT

```
Machine 1       Machine 2       Machine 3       Machine 4
   10 TB           10 TB           10 TB           10 TB
```

Now we want to store one huge file across all these machines.

That's where the idea of a **distributed file system** comes in.

> **Distributed file system = a file system whose data is distributed across multiple machines.**

HDFS = **Hadoop Distributed File System**.

The textbook describes HDFS as a file-system component of Hadoop designed to run on a **cluster of commodity hardware**, providing high-throughput access to large datasets.

Apache describes the same basic goal: HDFS is designed for large datasets on clusters of relatively inexpensive machines and is built to tolerate hardware failures.

---

2. What is a cluster?

You will hear this word constantly in Hadoop.

A **cluster** is simply a collection of computers working together.

For example:

TEXT

```
                HADOOP CLUSTER

              % % % % % % % % % % % % % % % %
              %        C o m p u t e r   1  %
              % % % % % % % % % % % % % % % %
                             %
              % % % % % % % % % % % % % % % %
              %        C o m p u t e r   2  %
              % % % % % % % % % % % % % % % %
                             %
              % % % % % % % % % % % % % % % %
              %        C o m p u t e r   3  %
              % % % % % % % % % % % % % % % %
                             %
              % % % % % % % % % % % % % % % %
              %        C o m p u t e r   4  %
              % % % % % % % % % % % % % % % %
```

Each computer is called a **node**.

So:

> **Node = one machine in the cluster.**

A Hadoop cluster might contain many nodes.

The textbook specifically says HDFS can have **thousands of DataNodes** and many clients connected simultaneously.

3. Why not just put everything on one huge machine?

There are several problems.

Problem 1 — Storage capacity

Suppose:

```
TEXT
Data = 100 TB
```

A single machine might not have enough storage.

With multiple machines:

```
TEXT
 M a c h i n e   A  !’   2 5   T B
 M a c h i n e   B  !’   2 5   T B
 M a c h i n e   C  !’   2 5   T B
 M a c h i n e   D  !’   2 5   T B

 T o t a l  !’   1 0 0   T B
```

Problem 2 — Hardware failure

This is even more important.

Suppose you store:

```
TEXT
100 TB
      !“
one machine
```

That machine's disk fails.

Potentially:

```
TEXT
 1 0 0   T B  !’   G O N E
```

HDFS assumes that hardware failures are **normal rather than exceptional** when you have thousands of machines.

So HDFS needs a way to survive machine failures.

We'll see that this is where **replication** comes in.

Problem 3 — Processing huge amounts of data

Suppose you have:

```
100 TB of data
```

and want to process it.

One computer has to read:

```
100 TB
```

But if the data is distributed:

```
M a c h i n e   1  !’   2 5   T B
M a c h i n e   2  !’   2 5   T B
M a c h i n e   3  !’   2 5   T B
M a c h i n e   4  !’   2 5   T B
```

then potentially:

```
M a c h i n e   1  % %
M a c h i n e   2  % %$
M a c h i n e   3  % %<% % !’   p r o c e s s   i n   p a r a l l e l
M a c h i n e   4  % %
```

This is one of the important ideas behind the Hadoop ecosystem.

The textbook says MapReduce was designed to handle parallelization, fault tolerance, data distribution, load balancing, etc., so programmers don't have to manually manage all those distributed-system problems.

But don't mix MapReduce with HDFS yet.

For now:

```
HDFS
  !“
STORAGE

MapReduce
  !“
PROCESSING
```

We'll get to MapReduce after HDFS.

4. So what exactly is HDFS?

Let's build the definition ourselves.

We have:

```
Huge data
      !“
Need multiple machines
```

```
      !“
Need distributed storage
      !“
Need to survive machine failures
      !“
Need efficient access to huge datasets
      !“
HDFS
```

Therefore:

> **HDFS is Hadoop's distributed file system. It stores large files across multiple machines in a cluster and uses replication to tolerate failures.**

The textbook emphasizes that HDFS is optimized for **high throughput and large datasets**, rather than low-latency interactive access.

That's an important characteristic.

---

5. HDFS is NOT just "a bigger hard disk"

This distinction is important.

Suppose you have:

TEXT

```
Computer A
Computer B
Computer C
```

HDFS makes these machines collectively behave like a distributed storage system.

From the user's/application's perspective, you can have:

TEXT

```
/user/data/sales.csv
```

rather than manually thinking:

TEXT

```
 C o m p u t e r   A  !’   p a r t   1
 C o m p u t e r   B  !’   p a r t   2
 C o m p u t e r   C  !’   p a r t   3
```

HDFS handles that organization.

The textbook says HDFS maintains a traditional hierarchical namespace of **files and directories**, similar conceptually to a normal file system.

---

6. The two most important types of machines

Now we're ready for the architecture.

HDFS has a **master-slave architecture** in the terminology used by your textbook.

There are two major roles:

TEXT

```
                HDFS

                    % % % % % % % % % % % % % % % %
                    %      N a m e N o d e        %
                    %          M A S T E R        %
                    % % % % % % % % %,% % % % % % % %
                                     %
               % % % % % % % % % % %<% % % % % % % % % %
               %                     %                  %
               %¼                    %¼                 %¼
   DataNode    DataNode    DataNode
    SLAVE       SLAVE       SLAVE
```

The textbook explicitly describes the NameNode as the master and DataNodes as the workers/slaves.

But don't memorize "master = important, slave = less important."

Understand **what each one actually does**.

---

7. DataNode — where the actual data lives

Think about a normal computer.

You have:

TEXT

```
Hard Disk
      !“
actual file data
```

In HDFS, the **DataNodes store the actual data**.

For example:

TEXT

```
DataNode 1
% % %    b l o c k s   o f   f i l e s

DataNode 2
% % %    b l o c k s   o f   f i l e s

DataNode 3
% % %    b l o c k s   o f   f i l e s
```

The textbook says application data is stored on DataNodes, while metadata is maintained separately by the NameNode.

So:

> **DataNode = machine that stores HDFS data blocks and serves read/write requests.**

DataNodes can also create, delete and replicate blocks when instructed by the NameNode.

---

8. NameNode — this is where people often get confused

The NameNode **does not normally store the actual contents of your huge files**.

Instead, it stores **metadata**.

What's metadata?

Information **about** the data.

For example, imagine:

```
TEXT
sales.csv
```

The actual contents might be:

```
TEXT
1001,Chennai,500
1002,Delhi,700
1003,Mumbai,400
...
```

That's **data**.

Metadata would be things like:

```
TEXT
File name: sales.csv
Owner: user1
Permissions: ...
Size: 500 GB
Blocks:
        B l o c k  1  !’  D a t a N o d e  3
        B l o c k  2  !’  D a t a N o d e  7
        B l o c k  3  !’  D a t a N o d e  2
```

So:

```
TEXT
               HDFS

       NameNode
              % % % % % % % % %
       METADATA
                    %
                    %    " W h e r e  i s  B l o c k  1 ? "
                    %    " W h e r e  i s  B l o c k  2 ? "
                    %
                    %¼
      % % % % % % % % % % % % % % %
      %      D a t a N o d e s    %
      %                           %
      %    a c t u a l   d a t a  %
      % % % % % % % % % % % % % % %
```

The textbook explicitly says:

> **metadata !’ NameNode application data !’ DataNodes.**

And this separation is a fundamental HDFS architectural idea.

9. Why separate them?

This is actually clever.

Imagine you want to read:

```text
sales.csv
```

The NameNode doesn't need to send you the entire file.

Instead:

```text
Client
    %
    %    " W h e r e   i s   s a l e s . c s v ? "
    %¼
NameNode
    %
    %    " I t s   b l o c k s   a r e   o n   D a t a N o d e   2 ,   5 ,   7 "
    %¼
Client
    %
    % % % % % % % % % !’   D a t a N o d e   2
    % % % % % % % % % !’   D a t a N o d e   5
    % % % % % % % % % !’   D a t a N o d e   7
```

The actual data travels **directly between the client and DataNodes**.

The NameNode primarily tells the client **where to find it**.

The textbook specifically describes this read process: the client first contacts the NameNode to obtain block locations, then connects directly to the DataNodes containing those blocks.

Apache's documentation similarly emphasizes that user data does not flow through the NameNode.

10. But how can one file be on multiple machines?

This brings us to **blocks**.

Suppose we have a huge file:

```text
movie.mp4
```

Size:

```text
500 GB
```

Instead of treating it as one giant object, HDFS divides the file into **blocks**.

Conceptually:

TEXT

```
movie.mp4

% % % % % % % % % %
%  B l o c k   1  %
% % % % % % % % % %$
%  B l o c k   2  %
% % % % % % % % % %$
%  B l o c k   3  %
% % % % % % % % % %$
%  B l o c k   4  %
% % % % % % % % % %$
%  B l o c k   5  %
% % % % % % % % % %
```

These blocks can be stored on different DataNodes:

TEXT

```
 D a t a N o d e   1  !’   B l o c k   1
 D a t a N o d e   2  !’   B l o c k   2
 D a t a N o d e   3  !’   B l o c k   3
 D a t a N o d e   4  !’   B l o c k   4
 D a t a N o d e   1  !’   B l o c k   5
```

The NameNode keeps track of the relationship:

TEXT

```
movie.mp4
      !“
 B l o c k   1  !’   D N 1
 B l o c k   2  !’   D N 2
 B l o c k   3  !’   D N 3
 B l o c k   4  !’   D N 4
 B l o c k   5  !’   D N 1
```

The textbook explicitly says that the NameNode uses metadata to determine the **mapping of file blocks to DataNodes**.

---

11. Why blocks?

Because now the system can distribute a huge file across machines.

Instead of:

TEXT

```
Machine A
% % % % % % % % %
500 GB file
```

we can have:

TEXT

```
 M a c h i n e   A  !’   B l o c k   1
 M a c h i n e   B  !’   B l o c k   2
 M a c h i n e   C  !’   B l o c k   3
 M a c h i n e   D  !’   B l o c k   4
```

This gives HDFS the ability to work with huge datasets across a cluster.

And there's another huge reason:

Replication.

Suppose:

```
TEXT
 B l o c k   1  !’   D a t a N o d e   1
```

and DataNode 1 dies.

Then:

```
TEXT
 B l o c k   1  !’   L O S T
```

unless we have another copy.

So HDFS creates multiple copies:

```
TEXT
Block 1
  % % %    D a t a N o d e   1
  % % %    D a t a N o d e   4
  % % %    D a t a N o d e   7
```

Now:

```
TEXT
 D a t a N o d e   1  Ø=Ü¥
```

doesn't necessarily mean:

```
TEXT
 B l o c k   1  Ø=Ü¥
```

because copies exist elsewhere.

The textbook states that the default replication factor discussed there is **three copies**, and replication provides reliability/high availability.

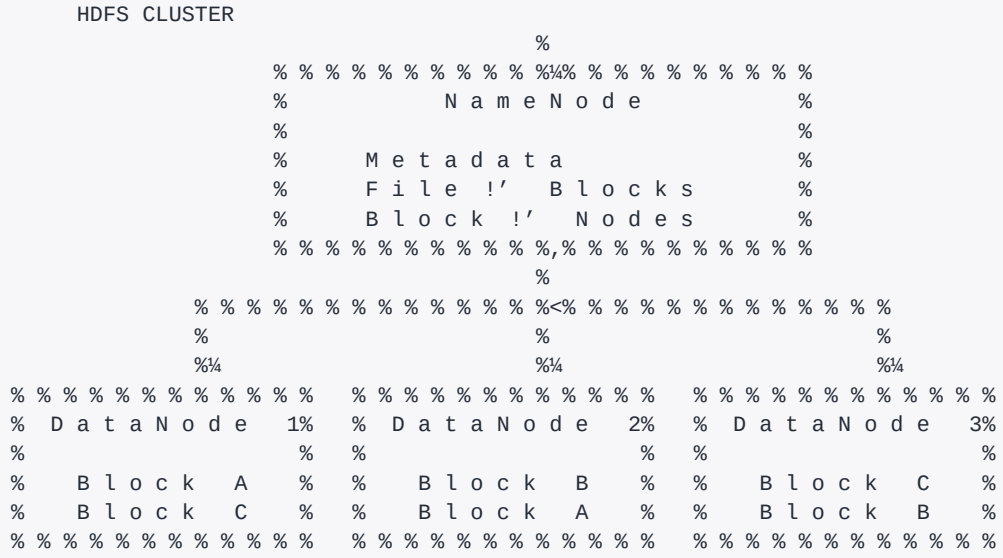
The exact default/configuration can vary by Hadoop version and configuration, so for understanding, think:

> **Replication factor = number of copies of a block that HDFS maintains.**

12. The architecture we have so far

Put everything together:

TEXT



Notice what is happening:

NameNode

Knows:

TEXT

```
Which files exist
Which blocks make up each file
Where those blocks are
File metadata
```

DataNodes

Actually store:

TEXT

```
Block A
Block B
Block C
...
```

Replication

Means the same block can exist on multiple DataNodes.

---

13. One subtle but VERY important distinction

Don't think:

> **"NameNode stores information and DataNode stores information."**

That's too vague.

Think:

```text
NameNode
      !“
"What is this file?"
"Where are its blocks?"
"What blocks should exist?"
"How many replicas?"
"Is a DataNode alive?"

DataNode
      !“
"Here is the actual block data."
"I can read it."
"I can write it."
"I can create/delete/replicate blocks when instructed."
```

The textbook describes DataNodes as serving client read/write requests and performing block operations under NameNode instructions.

Checkpoint 1 — don't look ahead

Answer these in your own words. **Don't worry about textbook wording.**

Q1

You have a 500 GB file in HDFS.

Why doesn't HDFS simply store the whole thing on one DataNode?

Q2

What is the difference between:

```text
NameNode
```

and

```text
DataNode
```

Q3

Suppose:

```text
sales.csv
      !“
Block 1
Block 2
Block 3
```

and the NameNode knows:

TEXT

```
Block 1 !’ DataNode A
Block 2 !’ DataNode B
Block 3 !’ DataNode C
```

Does the **NameNode send the actual contents of Block 1** to the client when the client wants to read it?

Q4

Why does HDFS make multiple copies of a block?

Once you answer these, we'll move to the next piece: **exactly how files are divided into blocks, how replication works, and why racks matter**.

USER

1. because we might not have that much storage on a single system , also storing on a single system is risky since it might be lost if the hardware fails 2. namenode stores metadata and datanode stores the actual data 3. no , namenode just sends the info about where the blocks exists to the client , the client fetches the data from the respective datanodes 4. to avoid loss of data one thing, i didnt read the textbook , i dont know anything , dont assume i know it , teach me everything you left out

ASSISTANT

Your answers are correct. And your clarification is important: **I was moving too quickly and assuming background knowledge you don't have.** From here, I'll teach this as if HDFS is completely new to you.

Also, we will follow the material you were given rather than assuming you already know Hadoop terminology. Your textbook introduces HDFS in §25.3 and then goes through architecture, I/O, replica management, and scalability. The Apache HDFS Architecture Guide you were given describes the same basic architecture.

We also **will not jump to blocks/replication and assume you know what they mean**. Let's build every concept in order.

Part 1 — First understand what a "file system" is

Before HDFS, let's understand the word **file system**.

You already use one every day.

For example, on Windows:

TEXT

```
C:\
%
% % %   U s e r s
%      % % %   S h r i s
%             % % %   D o c u m e n t s
%             % % %   D o w n l o a d s
%             % % %   P i c t u r e s
%
% % %   P r o g r a m   F i l e s
% % %   W i n d o w s
```

This is a **file system**.

A file system is the software/organization that allows a computer to:

- create files
- store files
- find files
- read files
- write files
- delete files
- organize files into directories

So if you do:

```text
C:\Users\Shris\Documents\notes.txt
```

the operating system needs to know:

> **Where is `notes.txt` physically stored on the disk?**

The file system manages that.

Part 2 — What changes when we have many computers?

With one computer:

```text
              COMPUTER

               % % % % % % % % % % % % % % % % % % % %
               %      F i l e   S y s t e m        %
               %                                   %
               %      n o t e s . t x t            %
               %      p h o t o . j p g            %
               %      d a t a . c s v              %
               % % % % % % % % % %,% % % % % % % % %
                                  %
              Disk
```

Everything is on one machine.

Now imagine we have:

```text
Computer 1
Computer 2
Computer 3
Computer 4
Computer 5
...
Computer 1000
```

and we want these computers to collectively store enormous amounts of data.

We need a **distributed file system**.

"Distributed" simply means:

> **The storage is spread across multiple computers.**

So:

```text
            DISTRIBUTED FILE SYSTEM

              C o m p u t e r   1  % % %
              C o m p u t e r   2  % % %$
              C o m p u t e r   3  % % %$
              C o m p u t e r   4  % % %<% % !’   o n e   s t o r a g e   s y s t e m
              C o m p u t e r   5  % % %$
                    . . .             %
              C o m p u t e r   N  % % %
```

HDFS is Hadoop's distributed file system.

The textbook specifically says HDFS is designed to run on a cluster of **commodity hardware**.

What's "commodity hardware"?

It means ordinary, relatively inexpensive computers rather than one extremely expensive specialized machine.

The basic philosophy is:

> **Instead of buying one enormous machine, use many ordinary machines and make them work together.**

Part 3 — Why do we need Hadoop in the first place?

Now we can understand the bigger picture.

Imagine a company has:

```text
100 TB of data
```

A normal database/computer setup might struggle with storing and processing it efficiently.

So we can use:

```text
         Hadoop cluster

       % % % % % % % % % %
       %  M a c h i n e 1%
       % % % % % % % % % %$
       %  M a c h i n e 2%
       % % % % % % % % % %$
       %  M a c h i n e 3%
       % % % % % % % % % %$
       %  M a c h i n e 4%
       % % % % % % % % % %$
       %  M a c h i n e 5%
       % % % % % % % % % %
```

There are two different problems:

Problem A — Where do we store all this data?

HDFS

```
HDFS
  !“
Distributed storage
```

Problem B — How do we process all this data?

Historically, **MapReduce**.

```
MapReduce
  !“
Distributed processing
```

That's why your textbook calls **MapReduce and HDFS the two core components of Hadoop**.

So keep this mental picture:

```
                 HADOOP

               % % % % % % % % % % % % % % % % % % % % % % %
               %                                           %
               %                H D F S                    %
               %             S T O R A G E                 %
               %                                           %
               % % % % % % % % % % % % % % % % % % % % % % %

               % % % % % % % % % % % % % % % % % % % % % % %
               %                                           %
               %          M a p R e d u c e                %
               %          P R O C E S S I N G              %
               %                                           %
               % % % % % % % % % % % % % % % % % % % % % % %
```

Later, we'll learn YARN, Hive, etc.

But **right now we only care about HDFS**.

Part 4 — What does "distributed storage" actually mean?

Suppose we have:

```
file = movie.mp4
size = 500 GB
```

Suppose a particular machine can only conveniently store 100 GB for our example.

We can't put the whole file there.

So we divide the file.

Conceptually:

```
movie.mp4
%
% % %    P a r t   1
% % %    P a r t   2
% % %    P a r t   3
% % %    P a r t   4
% % %    P a r t   5
```

and distribute them:

```
 M a c h i n e   1  !′   P a r t   1
 M a c h i n e   2  !′   P a r t   2
 M a c h i n e   3  !′   P a r t   3
 M a c h i n e   4  !′   P a r t   4
 M a c h i n e   5  !′   P a r t   5
```

HDFS does something similar, but it uses the term **block** rather than simply "part."

So:

> **A block is a chunk of a file stored by HDFS.**

The Apache architecture guide describes an HDFS file as a sequence of blocks.

Part 5 — Why doesn't HDFS just call them "parts"?

Because a block is a defined unit of storage in HDFS.

Imagine:

```
bigfile.txt

% % % % % % % % % % %,% % % % % % % % % % %,% % % % % % % % % % %,% % % % % % % % % %
%    B l o c k   1    %    B l o c k   2    %    B l o c k   3    %    B l o c k   4    %
% % % % % % % % % % %4% % % % % % % % % % %4% % % % % % % % % % %4% % % % % % % % % %
```

Each block can be stored on a different DataNode.

For example:

```
             bigfile.txt

               Blocks
                                 %
              % % % % % % % % % % %<% % % % % % % % % %
              %¼                 %¼                 %¼
    Block 1    Block 2    Block 3
              %                  %                  %
              %¼                 %¼                 %¼
     DN-1       DN-2       DN-3
```

And the **NameNode keeps track of this mapping**.

That is:

```
File
  !“
Blocks
  !“
DataNodes
```

Part 6 — What exactly is the NameNode keeping track of?

This is where the word **metadata** becomes important.

Suppose:

```
report.pdf
```

is a 300 MB HDFS file.

The actual content might be:

```
"Chapter 1..."
"Chapter 2..."
...
```

That is the **data**.

But we also need information such as:

```
Name: report.pdf
Owner: Shris
Permissions: ...
Size: 300 MB

Blocks:
    Block 1
    Block 2
    Block 3

Locations:
        B l o c k   1  !’   D a t a N o d e   1
        B l o c k   2  !’   D a t a N o d e   4
        B l o c k   3  !’   D a t a N o d e   2
```

This information **about the file** is metadata.

So:

> **Metadata = information that describes/manages the actual data.**

The textbook says the NameNode maintains the file-system namespace and metadata such as information about files/directories and the mapping of file blocks to DataNodes.

Part 7 — What's a namespace?

You will see this word in the textbook, so let's not leave it unexplained.

Think of your Windows folders:

```
TEXT
C:\
% % %   U s e r s
        % % %   S h r i s
                % % %   D o c u m e n t s
                        % % %   n o t e s . t x t
```

This hierarchy is the **file-system namespace**.

It's basically:

> **The organization and naming of files and directories.**

HDFS also has a hierarchical namespace:

```
TEXT
/
% % %   u s e r
%       % % %   s h r i s
%       %       % % %   d a t a . c s v
%       %       % % %   r e p o r t . t x t
%
% % %   p r o j e c t
        % % %   i n p u t
```

The Apache guide explicitly says HDFS supports a traditional hierarchical file organization where users/applications can create directories and store files in them.

So when the textbook says:

> **"NameNode manages the file system namespace"**

translate that into:

> **The NameNode keeps track of the HDFS files/directories and their metadata.**

Part 8 — Now let's understand the NameNode properly

We can now say:

NameNode

The NameNode is the central HDFS service responsible for managing the file system's metadata/namespace.

It knows things like:

```text
What files exist?
What directories exist?
What blocks make up each file?
Where are those blocks stored?
What are the file permissions?
How many replicas should exist?
```

The actual data is **not normally sent through the NameNode**.

Apache explicitly says:

> **user data never flows through the NameNode.**

That's a very important point.

Part 9 — DataNode

Now the other side.

A **DataNode** is where the actual blocks are stored.

Imagine:

```text
DataNode 1

% % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%    B l o c k   1                                      %
%    B l o c k   7                                      %
%    B l o c k   9                                      %
%    B l o c k   1 2                                    %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

Another:

```text
DataNode 2

% % % % % % % % % % % % % % % % % % % % % % % % % % % % %
%    B l o c k   2                                      %
%    B l o c k   4                                      %
%    B l o c k   8                                      %
% % % % % % % % % % % % % % % % % % % % % % % % % % % % %
```

So:

```text
NameNode
        %
        %    m e t a d a t a
        %¼
"Block 1 is on DN1"
"Block 2 is on DN2"
"Block 3 is on DN3"

DataNodes
```

```
        %
        % % %    a c t u a l   B l o c k   1
        % % %    a c t u a l   B l o c k   2
        % % %    a c t u a l   B l o c k   3
```

The textbook says DataNodes manage the storage attached to the machines on which they run and serve client read/write requests.

Part 10 — Why can't the client just ask DataNodes directly?

Because the client doesn't initially know where the blocks are.

Suppose:

TEXT

```
file.txt
```

has:

TEXT

```
Block 1
Block 2
Block 3
```

and they happen to be:

TEXT

```
 B l o c k   1  !’   D N 7
 B l o c k   2  !’   D N 3
 B l o c k   3  !’   D N 9
```

How does the client know this?

It asks the NameNode.

TEXT

```
Client
      %
      %    " W h e r e   i s   f i l e . t x t ? "
      %¼
NameNode
      %
      %    " B l o c k   1  !’   D N 7 "
      %    " B l o c k   2  !’   D N 3 "
      %    " B l o c k   3  !’   D N 9 "
      %¼
Client
```

Then:

TEXT

```
 C l i e n t  % % % % % % % % % !’   D N 7
        Block 1

 C l i e n t  % % % % % % % % % !’   D N 3
```

```
       Block 2

C l i e n t  % % % % % % % % % !’   D N 9
       Block 3
```

This is exactly the separation you correctly described earlier.

Part 11 — Now the really important problem: failure

Suppose:

```text
B l o c k   1  !’  D a t a N o d e   7
```

Then:

```text
D a t a N o d e   7  Ø=Ü¥
```

If that was the only copy:

```text
B l o c k   1  Ø=Ü¥
```

The file is incomplete.

HDFS is designed around the assumption that hardware failures happen regularly in large clusters. The Apache architecture guide explicitly describes failures as an expected condition when hundreds or thousands of machines are involved.

So HDFS needs **fault tolerance**.

Fault tolerance

This simply means:

> **The system continues functioning correctly even when some components fail.**

Part 12 — Replication

The solution is:

> **Keep multiple copies of each block.**

Suppose:

```text
Block 1
```

has replication factor 3.

Then:

```
Block 1
      %
      % % %    C o p y  !’   D a t a N o d e   1
      % % %    C o p y  !’   D a t a N o d e   4
      % % %    C o p y  !’   D a t a N o d e   7
```

Now:

```
 D a t a N o d e   4  Ø=Ü¥
```

We still have:

```
 D a t a N o d e   1  !’   B l o c k   1
 D a t a N o d e   7  !’   B l o c k   1
```

So the block isn't lost.

The textbook describes the default replication factor as three in its HDFS discussion, while the Apache documentation makes clear that the replication factor is configurable.

Part 13 — But there's another problem

Imagine we put all three copies on the same machine:

```
DataNode 1
% % %    B l o c k   1
% % %    B l o c k   1
% % %    B l o c k   1
```

That's obviously useless.

Machine fails:

```
 D a t a N o d e   1  Ø=Ü¥
```

All three copies disappear.

So replicas need to be distributed.

At minimum, conceptually:

```
 D a t a N o d e   1  !’   B l o c k   1
 D a t a N o d e   2  !’   B l o c k   1
 D a t a N o d e   3  !’   B l o c k   1
```

But HDFS goes further.

Part 14 — What's a rack?

A data center contains physical machines arranged in groups called **racks**.

Conceptually:

TEXT

```
            DATA CENTER

      Rack 1            Rack 2

      % % % % % % % % % % %              % % % % % % % % % % %
      %    S e r v e r   1  %              %    S e r v e r   4  %
      %    S e r v e r   2  %              %    S e r v e r   5  %
      %    S e r v e r   3  %              %    S e r v e r   6  %
      % % % % % % % % % % %              % % % % % % % % % % %
```

Machines in the same rack generally have network connectivity through the same rack-level networking equipment.

Now imagine:

TEXT

```
Block 1
  % % %    C o p y   1  !'   R a c k   1
  % % %    C o p y   2  !'   R a c k   1
  % % %    C o p y   3  !'   R a c k   1
```

If the **entire rack** has a networking/power failure, you could lose access to all replicas.

So HDFS considers rack placement when placing replicas.

The textbook explains that Hadoop clusters are typically spread across multiple racks and that HDFS tries to distribute replicas among racks to balance reliability with network cost.

Apache calls this **rack awareness**.

Part 15 — So now our architecture is much clearer

TEXT

```
                        HDFS

                                  % % % % % % % % % % % % % %
                                  %      N a m e N o d e    %
                                  %                         %
                                  %    M e t a d a t a      %
                                  %    N a m e s p a c e    %
                                  %    B l o c k            %
                                  %    l o c a t i o n s    %
                                  % % % % % % %,% % % % % % %
                                               %
                          % % % % % % % % % % % %<% % % % % % % % % % % %
                          %                    %                        %
                          %¼                   %¼                       %¼

                 % % % % % % % % %       % % % % % % % % %       % % % % % % % % %
                 %  D a t a N o d e%      %  D a t a N o d e%      %  D a t a N o d e%
                 %        1        %      %        2        %      %        3        %
                 %                 %      %                 %      %                 %
```

```
                % B l o c k   A %       % B l o c k   A %       % B l o c k   B %
                % B l o c k   B %       % B l o c k   C %       % B l o c k   C %
                % % % % % % % % %       % % % % % % % % %       % % % % % % % % %
```

The NameNode knows **about** the blocks.

The DataNodes contain the **actual blocks**.

Part 16 — There is one more thing we haven't discussed: how does the NameNode know DataNodes are alive?

This is important for understanding the architecture.

Imagine:

```text
NameNode thinks:

DataNode 7 is alive.
```

But actually:

```text
DataNode 7
          Ø=Ü¥
```

The NameNode needs to find out.

DataNodes periodically send a message to the NameNode called a **heartbeat**.

Think of it as:

```text
DataNode 7
        %
        %    " I ' m   a l i v e . "
        %
        %¼
NameNode
```

and then again:

```text
DataNode 7
        %
        %    " I ' m   s t i l l   a l i v e . "
        %¼
NameNode
```

The textbook calls this the **heartbeat mechanism**.

If the NameNode stops receiving heartbeats from a DataNode, it can determine that the DataNode is unavailable and take appropriate action.

Part 17 — What is a Block Report?

The heartbeat tells the NameNode roughly:

> **"I'm alive."**

But the NameNode also needs to know:

> **"What blocks do you currently have?"**

So DataNodes periodically send a **Block Report**.

For example:

```
TEXT
 D a t a N o d e   7  !’   N a m e N o d e

Heartbeat:
"I'm alive."

Block Report:
"I currently have:
 Block 1
 Block 7
 Block 12
 Block 19"
```

The textbook says the Block Report contains information about the blocks held by the DataNode.

So:

```
TEXT
Heartbeat
        !“
"I'm alive."

Block Report
        !“
"Here are the blocks I have."
```

This distinction is worth remembering.

Part 18 — What does the NameNode do with that information?

Suppose the desired replication factor is 3.

The NameNode knows:

```
TEXT
Block 5

Currently:
 D N 1  !’   c o p y
 D N 4  !’   c o p y

Desired:
3 copies
```

So:

```text
2 copies currently
3 copies desired
```

This is called **under-replication**.

The NameNode can instruct a DataNode to create another replica.

The textbook describes a background mechanism that monitors blocks needing replication and instructs DataNodes to create and distribute replicas.

So:

```text
NameNode
      %
      %    " B l o c k   5   n e e d s   a n o t h e r   r e p l i c a . "
      %¼
DataNode
      %
      % % %    c r e a t e s   r e p l i c a
```

This is how HDFS automatically recovers from some failures.

Part 19 — One more concept: checksums

Suppose a DataNode says:

> **"Here's Block 5."**

But the data has become corrupted.

We need a way to detect that.

HDFS uses **checksums**.

Very simplified:

```text
Original data
            !“
checksum calculation
            !“
Checksum
```

Later:

```text
Data read from disk
            !“
calculate checksum again
            !“
compare
```

If:

```text
stored checksum == calculated checksum
```

then the data is considered intact.

If:

```text
stored checksum != calculated checksum
```

then corruption has been detected.

The textbook says checksums are generated for block data and used during reads to detect corruption; when corruption is detected, HDFS can arrange for a healthy replica to be used/recreated.

Part 20 — We can now understand the entire basic HDFS story

Imagine you upload:

```text
movie.mp4
```

to HDFS.

Step 1 — HDFS divides it into blocks

```text
movie.mp4

Block 1
Block 2
Block 3
Block 4
```

Step 2 — HDFS stores blocks on DataNodes

```text
DN1 !’ Block 1
DN2 !’ Block 2
DN3 !’ Block 3
DN4 !’ Block 4
```

Step 3 — HDFS replicates them

For example:

```text
Block 1 !’ DN1, DN5, DN8
Block 2 !’ DN2, DN6, DN9
...
```

Step 4 — NameNode records metadata

TEXT

```
movie.mp4
      !“
 B l o c k   1  !’   D N 1 ,   D N 5 ,   D N 8
 B l o c k   2  !’   D N 2 ,   D N 6 ,   D N 9
...
```

Step 5 — DataNodes send heartbeats

TEXT

```
 D N 1  !’   N a m e N o d e :   a l i v e
 D N 2  !’   N a m e N o d e :   a l i v e
 D N 3  !’   N a m e N o d e :   a l i v e
```

Step 6 — DataNodes send Block Reports

TEXT

```
 D N 1  !’   " I   h a v e   B 1 ,   B 5 ,   B 9 . . . "
 D N 2  !’   " I   h a v e   B 2 ,   B 4 ,   B 8 . . . "
```

Step 7 — A machine fails

TEXT

```
 D N 5  Ø=Ü¥
```

The NameNode detects the failure.

It sees:

TEXT

```
Block 1

 D N 1  !’   c o p y   e x i s t s
 D N 5  !’   L O S T
 D N 8  !’   c o p y   e x i s t s
```

There are still two copies.

It can arrange for another replica to be created so the desired replication level is restored.

---

21. One important correction to our simplified picture

Earlier I said:

TEXT

```
NameNode
      !“
metadata
```

That's correct, but don't imagine the NameNode is continuously storing every piece of block data.

Think:

TEXT

```
                    NameNode
                                        %
                          % % % % % % % % %4% % % % % % % % %
                          %                                  %
          FILE METADATA   BLOCK LOCATIONS
                          %                                  %
                          %¼                                 %¼
                  " f i l e . c s v "             B 1  !’   D N 1 , D N 4
                  " r e p o r t . p d f "         B 2  !’   D N 2 , D N 7
```

while:

TEXT

```
DataNodes
        %
        % % %    a c t u a l   B 1   c o n t e n t s
        % % %    a c t u a l   B 2   c o n t e n t s
        % % %    a c t u a l   B 3   c o n t e n t s
        % % %    . . .
```

That separation is one of the **central ideas of HDFS**.

The textbook explicitly says that metadata and application data are stored separately.

---

22. What we've actually learned

At this point, you should know these concepts **from scratch**:

Concept	Meaning
File system	Software/organization for storing and managing files
Distributed file system	File system whose storage is distributed across machines
Hadoop	Ecosystem/framework historically centered around distributed storage and processing
HDFS	Hadoop's distributed file system
Cluster	Collection of machines working together
Node	A machine in the cluster
DataNode	Stores actual HDFS blocks
NameNode	Manages HDFS metadata and namespace
Metadata	Information about the data
Namespace	Organization of files/directories
Block	Chunk of an HDFS file
Replication	Keeping multiple copies of a block
Replication factor	Number of copies desired
Fault tolerance	Continuing despite component failures

Concept	Meaning
Rack	Physical grouping of machines/networking
Rack awareness	Considering rack location when placing data
Heartbeat	DataNode's periodic "I'm alive" message
Block Report	DataNode's report of the blocks it holds
Checksum	Mechanism used to detect data corruption

And there are still important parts of your requested topic that we **have not yet properly learned**:

1. **Exact block structure and block size**
2. **Complete HDFS architecture**
3. **File read flow — step by step**
4. **File write flow — step by step**
5. **Write pipeline**
6. **How replication happens during writing**
7. **What happens when a DataNode fails**
8. **Secondary NameNode**
9. **Checkpoint**
10. **Journal / edit log**
11. **NameNode restart/recovery**
12. **HDFS functions/operations**
13. **Why HDFS is optimized for batch/high throughput rather than low latency**
14. **Single-writer/multiple-reader model**
15. **Append behavior**
16. **Network locality and why HDFS tries to read from nearby replicas**
17. **How all of these fit together in one complete architecture**

We will do those **one at a time**, rather than dumping them all at you.

Checkpoint 2

Before we move to the next section, answer these:

1. What is the difference between a **file** and a **block** in HDFS?

2. Suppose:

```
TEXT
file A
  !“
Block 1
Block 2
```

and:

TEXT

```
B l o c k  1 !’  D a t a N o d e  1
B l o c k  2 !’  D a t a N o d e  2
```

Where is the **actual content** of Block 1?

3. What information does the NameNode need to know about Block 1?

4. What is the difference between a **heartbeat** and a **Block Report**?

5. Suppose Block 7 has replication factor 3, but currently only two copies exist. What does that situation mean, and who detects/manages it?

After this, we'll start **blocks properly**, including how a file is actually divided and then move into the **complete read operation**.

USER

1. file is a whole data unit , while a block is a division/part of the data unit 2. block 1 contains a part of the data of the file A 3. namenode needs to know the metadata of the block 1 like to which file it belongs , where its replications are stored ,etc 4 . heartbeat used to inform the namenode that the datanode is alive , while block report is used to indicate what blocks the datanode contains 5. this is called underreplication, namenode detects this and orders another datanode to create a replication of this

ASSISTANT

Exactly. Your understanding is solid. Let me tighten two points before we continue.

Checkpoint 2 — corrections

1. File vs block — correct

A **file** is the complete logical data unit from the user's perspective.

A **block** is a chunk of that file that HDFS stores separately.

For example:

TEXT

```
File A = 500 MB

        File A
      % % % % % % %,% % % % % %,% % % % % %,% % % % % %
      %    B 1    %    B 2    %    B 3    %    B 4    %
      % % % % % % %4% % % % % %4% % % % % %4% % % % % %
```

The exact block size depends on the HDFS configuration/version; we'll discuss that shortly.

---

2. Where is Block 1's actual data? — correct

If:

TEXT

```
File A
     !“
Block 1
```

```
      !“
DataNode 1
```

then the **actual bytes belonging to Block 1 are stored on DataNode 1**.

If Block 1 is replicated:

TEXT

```
             Block 1
            /   |   \
                    !“        !“        !“
         DN1   DN2   DN3
```

then all three DataNodes contain a copy of the block's data.

---

3. What does the NameNode know? — mostly correct

One important distinction:

The NameNode does **not need to store the actual contents of Block 1**.

It needs metadata such as:

TEXT

```
File A
      !“
Block 1
      !“
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
```

So the NameNode's job is essentially to maintain the **map**:

> **"Which blocks make up this file, and where can I find those blocks?"**

The textbook specifically describes the NameNode as maintaining the filesystem namespace and the mapping of file blocks to DataNodes.

---

4. Heartbeat vs Block Report — exactly right

Think:

TEXT

```
Heartbeat:
"Hey NameNode, I'm alive."

Block Report:
"Here is the list of blocks I currently have."
```

The textbook says DataNodes communicate with the NameNode through heartbeats and Block Reports. A Block Report contains information such as block ID, generation stamp, and length.

---

5. Under-replication — correct

Exactly:

```
TEXT
Required copies = 3
Actual copies   = 2

                !“

Under-replication

                !“

NameNode notices

                !“

NameNode instructs a DataNode

                !“

New replica created
```

The NameNode tracks replica count/location and maintains blocks needing replication in a priority structure; a background process instructs DataNodes to create replicas.

Now: What exactly is an HDFS block?

This is important because we've been saying **"file !’ blocks"**, but we haven't yet understood what that actually means physically.

Suppose we have:

```
TEXT
File: movie.mp4
Size: 500 MB
```

HDFS doesn't necessarily treat that as one giant object stored on one DataNode.

Instead, conceptually:

```
TEXT
movie.mp4
        %
        % % %    B l o c k   1
        % % %    B l o c k   2
        % % %    B l o c k   3
        % % %    . . .
```

Each block is stored independently on DataNodes.

And then replication adds another layer:

```
TEXT
                 movie.mp4
                                    %
                % % % % % % % % % % % %<% % % % % % % % % % % %
                !“                  !“                        !“
     Block 1      Block 2      Block 3
                %                   %                        %
          % % % %<% % %             % % % %<% % %            % % % %<% % %
```

```
          !“    !“    !“            !“    !“    !“            !“    !“    !“
    DN1 DN2 DN3  DN2 DN4 DN5  DN1 DN3 DN6
```

So there are **two different ideas** here:

Splitting

A large file is divided into blocks:

TEXT

```
File
  !“
Block 1
Block 2
Block 3
...
```

Replication

Each block can have multiple copies:

TEXT

```
Block 1
  % % %    D N 1
  % % %    D N 2
  % % %    D N 3
```

Don't confuse these.

Splitting "`replication.

Why doesn't HDFS just store the entire file on one DataNode?

Imagine:

TEXT

```
File = 1 TB
```

and suppose one DataNode stores the whole thing:

TEXT

```
             1 TB
                          !“
           DataNode
```

We've immediately lost several advantages of a distributed system.

Instead, HDFS can distribute the blocks:

TEXT

```
                  1 TB File
                                        !“
                % % % % % % % % % % % % %<% % % % % % % % % % % %
                !“                          !“                          !“
      Block 1      Block 2      Block 3 ...
```

```
              !“                          !“                          !“
       DN1          DN2          DN3
```

Now multiple machines contribute their storage.

And replication provides protection against machine failure.

This matches the textbook's design goals: HDFS is intended for very large files on clusters of commodity hardware, where hardware failure is expected to occur.

One subtle but VERY important thing

You might now think:

> **"If the file is split across DataNodes, then when I want the file, I have to ask the NameNode to give me each block."**

No.

The NameNode does **not** send the actual file data.

Remember:

TEXT

```
                NameNode
             (metadata only)
                                %
                 " B l o c k   1  !’   D N 1 ,   D N 2 "
                 " B l o c k   2  !’   D N 3 ,   D N 4 "
                 " B l o c k   3  !’   D N 1 ,   D N 5 "
                                %
                                %    l o c a t i o n s
                                !“
                Client
               /  |  \
                           !“      !“      !“
            DN1  DN3  DN1
           Block Block Block
             1     2     3
```

The client first contacts the NameNode to find out **where the blocks are**.

Then the client directly communicates with the DataNodes to obtain the actual data.

The textbook explicitly describes this separation.

This is one of the **most important ideas in HDFS architecture**.

Let's make sure this distinction is locked in

Suppose:

TEXT

```
File A
  % % %    B l o c k   1
  % % %    B l o c k   2
```

```
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
 B l o c k   2  !’   D N 2 ,   D N 4 ,   D N 5
```

The NameNode knows:

```text
File A
      !“
Block 1, Block 2

 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
 B l o c k   2  !’   D N 2 ,   D N 4 ,   D N 5
```

But the NameNode **doesn't contain**:

```text
"Hello, this is the actual content of Block 1..."
```

That content lives on the DataNodes.

Next: HDFS READ operation

Now we're ready to understand something much more useful:

> **What actually happens when a user asks HDFS to read a file?**

Let's take:

```text
File A
  % % %    B l o c k   1
  % % %    B l o c k   2
  % % %    B l o c k   3
```

with:

```text
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
 B l o c k   2  !’   D N 2 ,   D N 4 ,   D N 5
 B l o c k   3  !’   D N 1 ,   D N 3 ,   D N 6
```

The process starts with the **client**.

Step 1 — Client asks NameNode

```text
Client
      %
      %    " I   w a n t   F i l e   A "
      !“
NameNode
```

The NameNode looks up:

```
File A
  !“
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
 B l o c k   2  !’   D N 2 ,   D N 4 ,   D N 5
 B l o c k   3  !’   D N 1 ,   D N 3 ,   D N 6
```

It returns the **locations**.

Step 2 — Client contacts DataNodes directly

```
            Client
           /   |   \
                 !“        !“        !“
        DN1   DN2   DN1
        B1    B2    B3
```

The NameNode is no longer carrying the actual data.

Step 3 — Client reconstructs the file

The client gets:

```
Block 1
Block 2
Block 3
```

and presents them as:

```
File A
```

To the user, it looks like one normal file.

That's why we can say:

> **HDFS distributes the physical storage of a file while maintaining the illusion of one logical file.**

Checkpoint 3

Answer these before we move to **write flow**, which is more complicated:

1. If a client wants `File A`, does the NameNode send the actual contents of `File A` to the client?

2. Suppose:

```
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
```

Why are there three DataNodes listed?

3. If DN1 and DN2 both have Block 1, and the client wants Block 1, does it need to read from both?

4. Complete this:

TEXT

```
Client
      !“
NameNode
      !“
       ?
      !“
DataNode
      !“
Actual file data
```

What is the missing thing between NameNode and DataNode?

USER

1. no , it sends the information about where the node is stored 2. block is replicated , so three datanodes are listed 3. nope , read from any one of them 4. namenode->sends metadata->client->fetches the data from -> datanode

ASSISTANT

Yes— all four are correct. One small wording improvement for **#1**: the NameNode sends the **block locations (metadata)**, not simply "where the node is stored."

So the complete idea is:

TEXT

```
Client
    %
    %    " I   w a n t   F i l e   A "
    %¼
NameNode
    %
    %    " B l o c k   1  !’  D N 1 / D N 2 / D N 3
    %      B l o c k   2  !’  D N 2 / D N 4 / D N 5
    %      B l o c k   3  !’  D N 1 / D N 3 / D N 6 "
    %¼
Client
    %
    %    d i r e c t l y   r e q u e s t s   b l o c k s
    %¼
DataNodes
    %
    %¼
Actual data
```

This is exactly the metadata/data separation described in your textbook: the NameNode manages metadata, while DataNodes store and serve the actual application data.

And yes, for reading a replicated block, the client can choose **one available replica**. HDFS prefers a nearby replica to reduce network traffic.

Now let's learn the WRITE operation

Reading is relatively simple:

```
TEXT
 N a m e N o d e  !’  t e l l s  c l i e n t  w h e r e  d a t a  i s
 D a t a N o d e  !’  g i v e s  c l i e n t  d a t a
```

Writing is more interesting because HDFS has to decide:

- Where should the blocks go?
- How should replicas be created?
- How does the data move between DataNodes?
- How does HDFS ensure the data is successfully written?
- What happens if a DataNode fails during writing?

Let's build this from zero.

---

1. Suppose we want to create a file

The client wants to write:

```
TEXT
File A
```

The first thing it does is **not** send the file directly to a DataNode.

It first contacts the NameNode:

```
TEXT
Client
      %
      %    " I   w a n t   t o   c r e a t e   F i l e   A "
      %¼
NameNode
```

The NameNode manages the filesystem namespace, so it handles the creation of the file in the namespace and coordinates the storage locations.

---

2. The file is divided into blocks

Suppose our simplified example is:

```
TEXT
File A = 300 MB

        File A
                      %
            % % % % % %<% % % % %
            !“        !“        !“
     B1   B2   B3
```

Each block needs to be stored on DataNodes.

And suppose the replication factor is **3**:

```
B 1  !’   3   c o p i e s
B 2  !’   3   c o p i e s
B 3  !’   3   c o p i e s
```

Now the NameNode has to determine where those replicas should go.

3. The NameNode chooses DataNodes

For example:

```
B 1  !’   D N 1 ,   D N 2 ,   D N 3
B 2  !’   D N 2 ,   D N 4 ,   D N 5
B 3  !’   D N 1 ,   D N 3 ,   D N 6
```

Why not just put all three copies on DN1?

Because that would defeat much of the purpose of replication.

Imagine:

```
B1:
DN1
DN1
DN1
```

If DN1 fails:

```
D N 1  Ø=Ü¥
```

all three copies are gone.

Instead:

```
B1:
DN1
DN2
DN3
```

If DN1 fails:

```
D N 1  Ø=Ü¥

D N 2  !’   B 1  '
D N 3  !’   B 1  '
```

The textbook also discusses **rack-aware placement**, where replicas are distributed across racks to improve reliability and network behavior.

4. But how are the replicas actually created?

This is the important part.

Suppose the NameNode decides:

```
TEXT
 B 1  !’   D N 1 ,   D N 2 ,   D N 3
```

The client doesn't necessarily send three independent copies:

```
TEXT
 C l i e n t  % %  B 1% % !’   D N 1
 C l i e n t  % %  B 1% % !’   D N 2
 C l i e n t  % %  B 1% % !’   D N 3
```

Instead, HDFS uses a **write pipeline**.

Think of it as passing the data through a chain:

```
TEXT
Client
      %
      %    B 1
      %¼
  DN1
      %
      %    B 1
      %¼
  DN2
      %
      %    B 1
      %¼
  DN3
```

So:

Client !’ DN1 !’ DN2 !’ DN3

This is called a **pipeline** because the data flows through the DataNodes one after another.

Your textbook specifically describes HDFS writing data in packets through a write pipeline.

Why use a pipeline?

Imagine the client had to send the same block independently to three DataNodes:

```
TEXT
                                  % % % !’   D N 1
                                  %
 C l i e n t  % % % % % % % % % % %<% % !’   D N 2
                                  %
                                  % % % !’   D N 3
```

The client is responsible for sending the data three times.

With a pipeline:

```
TEXT
 C l i e n t  !’  D N 1  !’  D N 2  !’  D N 3
```

the client sends the data into the pipeline, and the DataNodes forward it along.

This is one reason the architecture can efficiently handle replicated writes.

5. What does the NameNode actually do here?

This distinction is extremely important.

The NameNode says, essentially:

> **"These are the DataNodes that should store the block."**

It **does not become part of the data pipeline**.

So:

```
TEXT
                METADATA
 C l i e n t  % % % % % % % % % % % % % % % % % % % % !’  N a m e N o d e
        "Where should B1 go?"
                             %
                             %
                             %¼
                    D N 1  !’  D N 2  !’  D N 3
             ACTUAL DATA
```

The actual file data does **not** flow:

```
TEXT
 C l i e n t  !’  N a m e N o d e  !’  D a t a N o d e
```

Instead:

```
TEXT
 C l i e n t  !’  D a t a N o d e  !’  D a t a N o d e  !’  D a t a N o d e
```

The NameNode handles the **control/metadata side**, while DataNodes handle the **actual data**.

This separation is a central feature of HDFS architecture.

6. What happens when Block 1 is finished?

Suppose:

```
TEXT
B1
!“
 D N 1  !’  D N 2  !’  D N 3
```

has been successfully written.

The DataNodes maintain information about their stored blocks and communicate with the NameNode using heartbeats and Block Reports.

The NameNode can therefore maintain its view of which DataNodes contain which blocks.

Then the process continues with the next block:

```
TEXT
File A

 B 1  !’   D N 1  !’   D N 2  !’   D N 3

 B 2  !’   D N 2  !’   D N 4  !’   D N 5

 B 3  !’   D N 1  !’   D N 3  !’   D N 6
```

Eventually the complete file has been stored.

One more important concept: "single writer, multiple readers"

Your textbook says HDFS follows a **single-writer/multiple-reader** model.

That means HDFS is designed around:

```
TEXT
Many clients can READ
                !“
      File
                !‘
   One writer
```

It does **not** use the same model as a normal filesystem where many different processes freely modify the same file simultaneously.

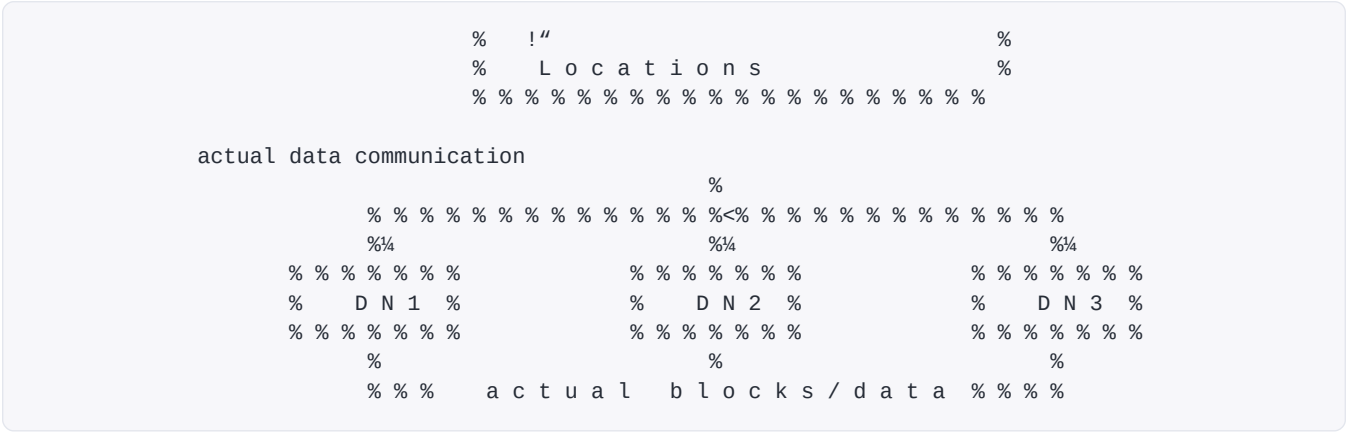
The textbook describes HDFS as having a simple coherency model: files have a single writer, and their contents cannot be updated arbitrarily; they can be appended to.

We'll come back to **append** because it is an important part of understanding why HDFS is designed this way.

Your mental model so far

You should now be able to picture HDFS as three layers:

```
TEXT
                                % % % % % % % % % % % % % % % % % % % %
                                %            C L I E N T              %
                                % % % % % % % % % %,% % % % % % % % % %
                                                  %
                  asks for metadata
                                                  %
                                                  %¼
                                % % % % % % % % % % % % % % % % % % % %
                                %          N A M E N O D E            %
                                %                                     %
                                %    F i l e s                          %
                                %   !“                                  %
                                %    B l o c k s                        %
```



And during a replicated write:

```text
Client
   %
   %   actual data
   %¼
 DN1
   %
   %   forward
   %¼
 DN2
   %
   %   forward
   %¼
 DN3
```

Checkpoint 4 — write flow

Answer these:

1. When writing a file, does the actual file data pass through the NameNode?

2. If the NameNode decides:

```text
Block 1 !' DN1, DN2, DN3
```

what is the write pipeline?

3. Why is:

```text
Client !' DN1 !' DN2 !' DN3
```

called a pipeline?

4. What is the NameNode responsible for during the write, and what are the DataNodes responsible for?

After this, we'll go into the **exact write pipeline**, including **packets, acknowledgements, checksums, `hflush`, and what happens when a DataNode fails during a write**. Those details are in your textbook and are the parts that usually make HDFS write flow confusing.

USER

1. no , namenode just tells where the blocks should be stored , that's all 2. client -> sends block data -> dn1 -> writes the data block and forwards -> dn2 -> writes and forwards -> dn3 , this is the write pipeline 3. because the data is forwarded 4. deciding the block splits , which blocks go to which nodes , the replication factor of each block , where those replications are stored , while the datanodes store the actual block data and forwards them to other datablock

ASSISTANT

Yes. Your write-flow understanding is correct. There is just **one important correction to #4**.

Checkpoint 4 — correction

Your pipeline:

```
Client
    !“
DN1
    !“
DN2
    !“
DN3
```

is correct. Each DataNode stores the incoming data and forwards it along the pipeline. Your textbook describes this as a write pipeline using packets.

For the NameNode, however, separate these responsibilities:

NameNode

It manages the **metadata and placement**:

- maintains the file/directory namespace
- maintains which blocks make up a file
- chooses/records the DataNodes where replicas should be placed
- keeps track of replica locations
- detects when blocks need additional replicas and instructs DataNodes to create them

The textbook describes the NameNode as maintaining the namespace and the mapping of file blocks to DataNodes.

It is better **not** to say that the NameNode "decides the block splits." The file is stored as blocks according to the HDFS block structure; the NameNode manages the metadata and placement of those blocks.

DataNode

DataNodes:

- store the **actual block data**
- serve read/write requests
- create/delete/replicate blocks when instructed
- participate in the write pipeline

And during a pipeline:

TEXT

```
Client
      %
      %    d a t a
      %¼
 DN1
      %
      %    f o r w a r d s   d a t a
      %¼
 DN2
      %
      %    f o r w a r d s   d a t a
      %¼
 DN3
```

One wording correction: DN1 forwards the **data/packets**, not "another datablock." A block is the larger storage unit; the data is transmitted in smaller **packets**.

Now let's understand packets

This is where the textbook gets more detailed.

We have been saying:

> **"Client sends Block 1 to DN1."**

That's conceptually correct, but physically the entire block isn't necessarily treated as one giant transmission.

The textbook says HDFS writes data in **64-KB packets** in a write pipeline.

So imagine:

TEXT

```
Block 1
%
% % %    P a c k e t   1
% % %    P a c k e t   2
% % %    P a c k e t   3
% % %    P a c k e t   4
% % %    . . .
```

The pipeline then looks more like:

TEXT

```
               Packet 1
 C l i e n t  % % % % % % % % % % % % % % % % % % !’   D N 1
                                                        %
                                                        !“
                           DN2
                                                        %
                                                        !“
                           DN3
```

Then:

TEXT

```
                Packet 2
 C l i e n t  % % % % % % % % % % % % % % % % % % !’   D N 1
                                                        %
                                                        !“
                              DN2
                                                        %
                                                        !“
                              DN3
```

and so on.

So don't imagine:

TEXT

```
 C l i e n t  !’   D N 1
        "Here is the entire 128 MB block."
```

Instead, conceptually:

TEXT

```
 C l i e n t  !’   p a c k e t  !’   D N 1  !’   D N 2  !’   D N 3
 C l i e n t  !’   p a c k e t  !’   D N 1  !’   D N 2  !’   D N 3
 C l i e n t  !’   p a c k e t  !’   D N 1  !’   D N 2  !’   D N 3
...
```

The **64-KB packet size is from the Hadoop version/textbook material you're studying**, so keep that value for your course rather than assuming it is a universal value for every Hadoop version.

Why packets?

Because a block can be large.

Suppose:

TEXT

```
Block = 64 MB
```

It would be inefficient to think of the entire thing as one indivisible transmission.

Instead:

TEXT

```
64 MB block
          !“
many smaller packets
          !“
pipeline
```

This also allows HDFS to keep track of the successful transmission of pieces of the data.

And this leads us to the next important concept:

Checksums

Suppose the client sends:

```
Packet 1
```

to DN1.

How do we know that the data wasn't corrupted while being transmitted or stored?

HDFS uses a **checksum**.

A checksum is essentially a calculated value associated with some data that allows the system to detect whether that data has changed or become corrupted.

Very simplified:

```
Original data
          !“
checksum calculation
          !“
Checksum = X
```

Later, the data can be checked again:

```
Data
  !“
calculate checksum
  !“
Result = X ?
```

If the expected and calculated values don't match:

```
Expected:  X
Actual:    Y

                !“

Possible corruption
```

Your textbook specifically mentions checksums in the HDFS write/read discussion and explains that corrupt blocks can be detected and another replica can be used.

Now the write pipeline becomes more complete

Instead of the simplified:

```
 C l i e n t  !’  D N 1  !’  D N 2  !’  D N 3
```

think:

```
                    Packet
                                        %
```



Each DataNode receives the data and participates in storing the replica.

The NameNode is **outside this data path**:

TEXT



That distinction is worth memorizing:

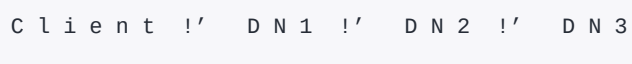
> **NameNode = tells you where. DataNodes = handle the actual data.**

But how does the client know when the pipeline succeeded?

This is where **acknowledgements (ACKs)** come in.

Suppose:

TEXT



The client sends a packet.

DN3 successfully receives it.

Then an acknowledgement can travel back through the pipeline:

TEXT



```
DN3
```

Conceptually, the client eventually learns:

> **"This packet has successfully reached the required replicas."**

So the overall communication is roughly:

TEXT

```
                          D A T A  !'
C l i e n t  % % % % % % % !'   D N 1  % % % % % !'   D N 2  % % % % % !'   D N 3

                         !•   A C K
C l i e n t  !•% % % % % %    D N 1  !•% % % % % %    D N 2  !•% % % % % %    D N 3
```

This is an important part of understanding the pipeline: **data moves forward; acknowledgements come back.**

What if DN2 fails?

This is where HDFS becomes interesting.

Suppose:

TEXT

```
C l i e n t  !'   D N 1  !'   D N 2  !'   D N 3
               X
             failure
```

DN2 suddenly fails.

We don't want the whole file write to simply become useless.

HDFS is designed around the expectation that hardware failures can occur.

The pipeline can be reconstructed using the remaining/available DataNodes, and HDFS can later restore the required replication level.

The exact failure-handling mechanics are more detailed than the basic architecture, so we'll walk through them rather than memorizing a vague "it handles failures."

Checkpoint 5

Before we go into **DataNode failure + pipeline recovery**, make sure these are clear:

1. If a 64-KB packet belongs to Block 1, is that packet itself a block?

2. In:

TEXT

```
C l i e n t  !'   D N 1  !'   D N 2  !'   D N 3
```

which direction does the **actual data** travel?

3. Which direction do the **ACKs** travel?

4. Why does HDFS use checksums?

5. If DN2 fails during:

```
TEXT
C l i e n t  !’  D N 1  !’  D N 2  !’  D N 3
```

does that mean the entire file is immediately lost?

USER

1. no 2. c->dn1->dn2->dn3 3. dn3->dn2->dn1->c 4. to ensure that the data is not corrupted during the transmission 5. no , we are just sending a packet now what are left ?

ASSISTANT

Correct on 1–4. For #**5**, your answer has the right intuition, but there's one distinction:

> **DN2 failing while writing one packet does not mean the entire file is lost. HDFS can detect the pipeline problem and reconstruct the pipeline; the already-written data can remain on the surviving DataNodes, and HDFS can restore the required replication.**

What is left?

We've covered quite a lot. Based specifically on your textbook's HDFS section, here's the remaining material we should learn.

' Already covered

1. What HDFS is
2. Why distributed storage is needed
3. Cluster and nodes
4. File vs block
5. Metadata
6. Namespace
7. NameNode
8. DataNode
9. NameNode !” DataNode relationship
10. Replication
11. Under-replication
12. Heartbeat
13. Block Report
14. Checksums
15. Rack awareness — basic idea
16. HDFS read flow — basic
17. HDFS write flow — basic
18. Write pipeline
19. Packets

20. ACKs

Ø=Ź4ll left

1. Complete HDFS write process

We know the simplified pipeline, but we haven't yet put **all pieces together**:

TEXT

```
Client
    !“
NameNode
    !“
DataNode selection
    !“
Pipeline construction
    !“
Packets
    !“
Checksums
    !“
ACKs
    !“
Block completed
```

We'll do this as one complete example.

2. DataNode failure during writing

We'll specifically see:

TEXT

```
C l i e n t  !’  D N 1  !’  D N 2  !’  D N 3
              X
            failure
```

What happens to:

- the pipeline
- the packet
- the replicas
- the remaining write
- replication afterward

This is important for understanding **fault tolerance** rather than just memorizing "HDFS handles failures."

3. DataNode failure after writing

Different situation:

TEXT

```
B 1  !’  D N 1 ,  D N 2 ,  D N 3

DN2 fails
```

Now:

TEXT

```
B 1  !’  D N 1 ,  D N 3
```

The block becomes **under-replicated**.

We'll connect this to:

TEXT

```
Heartbeat
        !“
NameNode detects failure
        !“
Block needs replication
        !“
NameNode chooses another DataNode
        !“
New replica
```

---

4. Rack awareness in detail

We've only covered the idea.

We'll understand:

TEXT

```
Rack 1                Rack 2
% % % % % % % % % % % %                  % % % % % % % % % % % %
%    D N 1             %                  %    D N 4             %
%    D N 2             %                  %    D N 5             %
%    D N 3             %                  %    D N 6             %
% % % % % % % % % % % %                  % % % % % % % % % % % %
```

Why putting all replicas in one rack is dangerous, and how HDFS considers **network distance** when choosing replica locations. The textbook discusses reliability, availability and aggregate bandwidth in relation to rack placement.

---

5. Read flow — detailed

We know:

TEXT

```
C l i e n t  !’  N a m e N o d e
N a m e N o d e  !’  b l o c k  l o c a t i o n s
C l i e n t  !’  D a t a N o d e s
```

But we'll still cover:

- choosing the nearest replica
- why the NameNode doesn't send data
- what happens if one replica is corrupt
- why reading locally is preferable

The textbook explicitly notes that HDFS prefers the nearest DataNode for reads.

---

6. NameNode's persistent metadata

This is a major section we haven't covered yet.

You need to understand:

```
NameNode
      %
      % % %   f i l e s y s t e m   i m a g e
      %
      % % %   j o u r n a l   /   e d i t   l o g
      %
      % % %   b l o c k - l o c a t i o n   i n f o r m a t i o n
```

And especially:

What happens when the NameNode restarts?

The textbook explains that the NameNode restores a checkpoint and then applies the journal to recover the current filesystem state.

This is an important part of HDFS architecture.

---

7. Secondary NameNode

This is **not a backup NameNode in the simple sense**.

We'll learn:

```
NameNode
        %
        % % %   c h e c k p o i n t
        %
        % % %   j o u r n a l
                      !“
   Secondary NameNode
                      !“
      checkpoint
```

Your textbook distinguishes the **Checkpoint Node** and **Backup Node**, and explains the Secondary NameNode's checkpointing role.

This is commonly misunderstood, so we'll take it slowly.

---

8. Checkpoint

We'll answer:

> **Why can't the NameNode just keep accumulating its journal forever?**

and understand:

```
Old checkpoint
       +
Journal
               !“
New checkpoint
```

9. HDFS file semantics

Your textbook has some important design assumptions:

```text
Single writer
Multiple readers

Write once
Read many times

 U p d a t e  !'  n o t  s u p p o r t e d  n o r m a l l y
 A p p e n d  !'  s u p p o r t e d
```

We'll understand **why HDFS chose this model**, rather than simply memorizing it.

10. `hflush`

Your textbook mentions `hflush`.

We'll understand what problem it solves and how it relates to writing data and making written data visible.

11. HDFS functions / operations

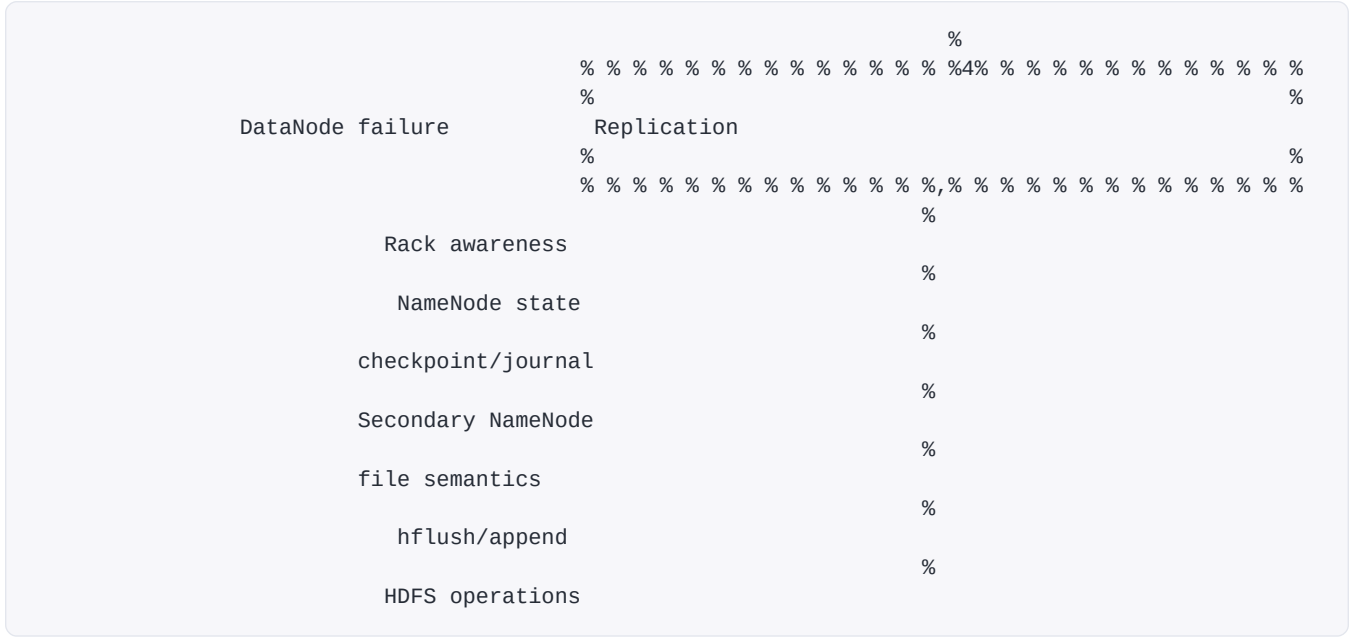
Finally we'll organize the things HDFS can do:

```text
Create
Read
Write
Append
Delete
Replicate
Recover
```

and connect each operation to the NameNode/DataNode roles.

So the remaining path is

```text
                  HDFS
                                    %
            % % % % % % % % % % % % % %4% % % % % % % % % % % % % %
            %                                                     %
    READ                          WRITE
            %                                                     %
      b a s i c  '                                          b a s i c  '
      d e t a i l e d  !•                                   d e t a i l e d  !•
                                                                  %
                                                        p i p e l i n e  '
                                                                  %
                                                        p a c k e t s  '
                                                                  %
                                                          A C K s  '
                                                                  %
                                                f a i l u r e  h a n d l i n g  !•
```



So we're **well past the basic architecture**, but there are still several important pieces before I would consider the HDFS Architecture portion complete.

Next I'd do **DataNode failure during the write pipeline**, because it completes the write flow we just started.

USER

ok , continue

ASSISTANT

Good. Let's continue with the next piece: **what happens when a DataNode fails during a write?**

This is where replication, the pipeline, ACKs, and fault tolerance all come together.

---

1. Start with a normal pipeline

Suppose the NameNode has selected:

```
TEXT
B l o c k  1 !’  D N 1 ,  D N 2 ,  D N 3
```

So the write pipeline is:

```
TEXT
Client
      %
      %    p a c k e t
      %¼
 DN1
      %
      %¼
 DN2
      %
      %¼
```

```
DN3
```

The client sends packets one after another.

For a successful packet:

```
TEXT
DATA
 C l i e n t  % % % !’   D N 1  % % % !’   D N 2  % % % !’   D N 3

ACK
 C l i e n t  !•% % %    D N 1  !•% % %    D N 2  !•% % %    D N 3
```

The ACK tells the upstream node that the packet reached the downstream part of the pipeline successfully.

---

2. Now DN2 fails

Imagine we're halfway through writing:

```
TEXT
 C l i e n t  !’   D N 1  !’   D N 2  !’   D N 3
                                 'L
             failure
```

What has happened?

DN2 is no longer available to continue the pipeline.

But notice something important:

```
TEXT
DN1
```

and

```
TEXT
DN3
```

may still be alive.

So this does **not automatically mean that the file has been lost**.

This is exactly the kind of failure HDFS is designed around: the textbook explicitly assumes that hardware failures are normal rather than exceptional.

---

3. What about the data already written?

Suppose packets 1–100 were successfully written before DN2 failed.

We might have:

```
Packet 1     !'  DN1 '   DN2 '   DN3 '
Packet 2     !'  DN1 '   DN2 '   DN3 '
...
Packet 100   !'  DN1 '   DN2 '   DN3 '
```

Those successfully written copies don't magically disappear just because DN2 subsequently fails.

So the failure affects the **current pipeline**, not necessarily all previously written data.

4. The pipeline has to be changed

Originally:

```
Client !'  DN1 !'  DN2 !'  DN3
```

DN2 is dead:

```
Client !'  DN1 !' 'L !'  DN3
```

The system can reconstruct the pipeline using the available DataNodes.

The important concept is:

> **A DataNode failure during writing causes the write pipeline to be reconstructed rather than simply abandoning the entire file.**

So conceptually:

```
Before:

Client !'  DN1 !'  DN2 !'  DN3


DN2 fails:


Client !'  DN1 !'  DN3
```

The exact low-level failure-handling mechanics depend on the Hadoop implementation/version; for your course, the important architectural idea is the **pipeline recovery**.

5. But now there's a replication problem

Originally:

```
Block 1
%%%   DN1 '
```

```
% % %    D N 2  '
% % %    D N 3  '

Replication factor = 3
```

DN2 fails:

```TEXT
Block 1
% % %    D N 1  '
% % %    D N 2  'L
% % %    D N 3  '
```

Now there are only **2 available replicas**.

Therefore:

```TEXT
Required = 3
Available = 2

!’   U n d e r - r e p l i c a t e d
```

And you already know what happens next.

The NameNode eventually identifies that the block needs another replica and instructs a DataNode to create one. The textbook describes the NameNode maintaining information about replica counts/locations and a background process instructing DataNodes to create missing replicas.

So:

```TEXT
DN2 fails
      !“
2 replicas remain
      !“
Under-replication
      !“
NameNode detects/records it
      !“
Another DataNode is selected
      !“
New replica created
```

Eventually:

```TEXT
Block 1
% % %    D N 1  '
% % %    D N 3  '
% % %    D N 5  '
```

Back to:

```TEXT
3 replicas
```

6. Notice the two different mechanisms

This is **very important**.

When DN2 fails, there are actually **two related but different problems**.

Problem A — current write

TEXT

```
Client !’  DN1 !’  DN2 !’  DN3
                      'L
```

The **pipeline needs to recover** so the client can continue writing.

Problem B — long-term replication

TEXT

```
DN1 '
DN2 'L
DN3 '

2 copies instead of 3
```

The **NameNode needs to restore the desired replication level**.

So don't mix them up:

TEXT

```
DataNode failure
          %
          %%%!’  Pipeline recovery
          %           "Can I continue writing?"
          %
          %%%!’  Replication recovery
           "Do I still have enough copies?"
```

That's a useful mental model.

7. What if a DataNode fails when we're NOT writing?

This is simpler.

Suppose the file was already completely stored:

TEXT

```
Block 1 !’  DN1, DN2, DN3
```

Then DN2 crashes:

TEXT

```
Block 1 !’  DN1, DN3
              '     '
```

The client isn't currently writing anything.

So there's no write pipeline to reconstruct.

The main issue is now:

> **"We have lost one replica."**

The NameNode can identify that the block needs replication and arrange for another copy.

```text
Block 1
% % %    D N 1  '
% % %    D N 2  'L
% % %    D N 3  '
             !“
   under-replicated
             !“
   new replica
             !“
           D N 5  '
```

This is why **replication is not just about surviving a failure at the exact moment of a write**.

It also protects already-stored data against later machine failures.

---

8. Why does the NameNode know DN2 failed?

Remember the **heartbeat**.

Normally:

```text
DN2
  %
  %    h e a r t b e a t
  !“
NameNode

"Still alive."
```

If DN2 stops responding:

```text
DN2
  %
  %   'L   n o   h e a r t b e a t
  !“
NameNode
```

The NameNode can determine that the DataNode is no longer available.

This is one reason heartbeats are important.

Remember:

```text
Heartbeat
    =
"I'm alive."
```

while:

```text
Block Report
    =
"Here are the blocks I currently have."
```

The textbook describes both as part of the NameNode/DataNode communication mechanism.

9. Now connect everything we've learned

Suppose:

```text
File A
    !“
Block 1
    !“
Replication factor = 3
    !“
DN1, DN2, DN3
```

During writing:

```text
Client
      %
      %    a s k s   N a m e N o d e
      %¼
NameNode
      %
      %    d e t e r m i n e s   p l a c e m e n t
      %¼
 C l i e n t  !’   D N 1  !’   D N 2  !’   D N 3
                       D A T A  !’

                      !•   A C K
```

If DN2 fails:

```text
 C l i e n t  !’   D N 1  !’   D N 2  'L  !’   D N 3
                   %
                   % % %    p i p e l i n e   r e c o n s t r u c t e d
```

Afterwards:

```text
Block 1
% % %    D N 1  '
% % %    D N 2  'L
% % %    D N 3  '

                !“

Under-replicated
```

```
                !“

NameNode

                !“

New replica

                !“

Block 1
% % %    D N 1  '
% % %    D N 3  '
% % %    D N 5  '
```

That's the whole fault-tolerance story.

Next: Rack Awareness

Now we're going to add **physical location** to this picture.

So far we've been saying:

TEXT

```
 B 1  !’   D N 1 ,   D N 2 ,   D N 3
```

But we haven't asked:

> **Where are DN1, DN2 and DN3 physically located?**

Suppose:

TEXT

```
                DATA CENTER

       Rack 1                  Rack 2
      % % % % % % % % % % % %                  % % % % % % % % % % % %
      %    D N 1              %                %    D N 4              %
      %    D N 2              %                %    D N 5              %
      %    D N 3              %                %    D N 6              %
      % % % % % % % % % % % %                  % % % % % % % % % % % %
```

A **rack** is a physical group of machines connected through networking equipment.

This matters because a whole rack can potentially become unavailable.

For example:

TEXT

```
Rack 1
      !“
network/power problem
      !“
 D N 1  'L
 D N 2  'L
 D N 3  'L
```

If all three replicas were there:

TEXT

```
Block 1
% % %    D N 1  'L
% % %    D N 2  'L
% % %    D N 3  'L
```

then replication didn't help much.

But if replicas are distributed:

TEXT

```
Rack 1              Rack 2

 D N 1  !•   B 1                        D N 4
 D N 2                                  D N 5  !•   B 1
 D N 3                                  D N 6  !•   B 1
```

then losing Rack 1 still leaves a copy on Rack 2.

The textbook specifically discusses rack organization, network distance, and block-placement policies in terms of reliability, availability, and network traffic.

We'll do **rack-aware placement properly next**, because there are several details worth understanding rather than memorizing "replicas go on different racks."

USER

continue

ASSISTANT

Let's continue with **rack awareness**, staying with the terminology and details from your textbook.

Rack awareness

We already know that HDFS stores replicas of a block on different DataNodes.

For example:

TEXT

```
Block 1
% % %    D N 1
% % %    D N 2
% % %    D N 3
```

But DataNodes aren't just an abstract list. In a real cluster, the machines are physically organized into **racks**.

A simplified layout:

TEXT

```
             Data Center

        Rack 1              Rack 2
```

```
          % % % % % % % % % % % % %                 % % % % % % % % % % % % %
          %     D N 1               %               %     D N 4               %
          %     D N 2               %               %     D N 5               %
          %     D N 3               %               %     D N 6               %
          % % % % % % % % % % % % %                 % % % % % % % % % % % % %
```

So each DataNode belongs to some rack.

Why does HDFS care about racks?

Because a **machine failure** isn't the only possible failure.

Imagine:

TEXT

```
Rack 1
      %
      % % %    D N 1
      % % %    D N 2
      % % %    D N 3
```

Something could happen to the whole rack—for example, a networking or power failure.

Then:

TEXT

```
 D N 1  'L
 D N 2  'L
 D N 3  'L
```

If all three replicas of a block were there:

TEXT

```
Block 1
% % %    D N 1  'L
% % %    D N 2  'L
% % %    D N 3  'L
```

we could lose access to every replica.

So HDFS considers **rack location when placing replicas**.

The textbook describes this as a **rack-aware** block placement policy and connects it to reliability, availability, aggregate read bandwidth, and network traffic.

Example

Suppose we have:

TEXT

```
Rack 1                Rack 2

DN1                   DN4
DN2                   DN5
DN3                   DN6
```

and:

```text
Replication factor = 3
```

Instead of:

```text
Block 1:

 D N 1  % %    R a c k   1
 D N 2  % %    R a c k   1
 D N 3  % %    R a c k   1
```

HDFS can distribute replicas across racks, conceptually:

```text
Block 1:

 D N 1  % %    R a c k   1
 D N 2  % %    R a c k   1
 D N 5  % %    R a c k   2
```

Now if Rack 1 disappears:

```text
 R a c k   1  'L

 D N 1  'L
 D N 2  'L

Rack 2

 D N 5  '
```

At least one replica remains available.

But there is a trade-off

You might think:

> **"Then why not always put every replica on a different rack?"**

Because **network distance matters**.

Suppose the client is connected to Rack 1.

Reading from:

```text
 C l i e n t  !’   D N 1
```

might require less network traffic than:

TEXT

```
C l i e n t  !’  R a c k  2  !’  D N 5
```

So HDFS tries to balance two goals:

TEXT

```
                Replica placement
                                        %
                        % % % % % % % % % %4% % % % % % % % % %
                        !“                                    !“
        Reliability          Network efficiency
        across racks          / locality
```

The textbook explicitly discusses minimizing network traffic while also providing reliability and availability.

Network distance

HDFS considers how "far away" machines are from one another in the network.

For example, conceptually:

TEXT

```
Same machine
          !“
Very close

Same rack
          !“
Nearby

Different rack
          !“
Farther

Different data center
          !“
Much farther
```

The exact network topology is represented so HDFS can make placement and read decisions based on locality.

This is why you previously learned:

> **When reading, HDFS prefers a nearby replica.**

For example:

TEXT

```
             Client
            /      \
                !“                !“
        DN1         DN5
       nearby       far away

                    !“
       Prefer DN1
```

This reduces unnecessary network traffic.

Now connect rack awareness to writing

Suppose the NameNode needs to place:

```
TEXT
Block 1
Replication factor = 3
```

It has to consider:

```
TEXT
1. Which DataNodes are available?
2. Where are those DataNodes?
3. How should replicas be distributed?
4. How can network traffic be minimized?
```

The result is a placement such as:

```
TEXT
             Block 1
            /   |   \
                  !“        !“        !“
         DN1   DN2   DN5
                    %           %           %
       Rack1  Rack1  Rack2
```

Then the write pipeline can be established among the selected DataNodes.

One subtle point: rack awareness "`"one replica per rack"

Don't memorize:

> **"Replication factor 3 means three different racks."**

That's too simplistic.

Rack-aware placement is a **placement policy** that considers rack topology and balances reliability with network efficiency. The exact placement depends on the cluster topology and Hadoop's placement policy.

For your course, the important conceptual chain is:

```
TEXT
Replication
          !“
Need multiple DataNodes
          !“
DataNodes have physical locations
          !“
Rack failures are possible
          !“
Spread replicas appropriately
          !“
Improve fault tolerance
```

Now let's revisit reading with rack awareness

Suppose:

```text
Block 1
% % %   D N 1   –   s a m e   r a c k   a s   c l i e n t
% % %   D N 2   –   s a m e   r a c k
% % %   D N 5   –   d i f f e r e n t   r a c k
```

The client wants Block 1.

It doesn't need:

```text
DN1 + DN2 + DN5
```

It needs **one replica**.

So HDFS can prefer a nearby one:

```text
Client
    %
    % % % !’   D N 1   '
    %
    % % % !’   D N 2
    %
    % % % !’   D N 5
```

The other replicas are there primarily for **fault tolerance and availability**, not because the client normally reads all three copies.

This is a very important distinction:

> **Replication provides multiple choices; a normal read does not require reading every replica.**

Where we are now

You now have the main **physical storage side** of HDFS:

```text
                    HDFS
                                    %
                          % % % % % % % % %4% % % % % % % % %
                          !“                                  !“
          NameNode          DataNodes
          metadata          actual data
                                                              %
                                                 % % % % % % % %4% % % % % % %
                                                 !“                            !“
                     Blocks       Replicas
                                                                               %
                                Rack-aware
```

```
placement
```

The next major piece is the **NameNode's own state**, because we've been treating the NameNode as if its metadata somehow magically stays there forever.

It doesn't.

We need to understand:

TEXT

```
NameNode
      %
      % % %    f i l e s y s t e m   i m a g e
      %
      % % %    j o u r n a l
      %
      % % %    c h e c k p o i n t
```

and then:

What happens if the NameNode itself crashes or restarts?

That's the next important section of your textbook.

USER

continur

ASSISTANT

Let's continue with the **NameNode's persistent state**. This is an important part because we've been treating the NameNode as the place where all the metadata lives, so we now need to answer:

> **How does the NameNode preserve that metadata, especially if it restarts?**

Your textbook covers this in the section immediately after the basic NameNode/DataNode architecture.

---

1. What information does the NameNode need to remember?

Recall our example:

TEXT

```
File A
  % % %    B l o c k   1
  % % %    B l o c k   2
  % % %    B l o c k   3
```

The NameNode needs information about the filesystem, such as:

TEXT

```
File A
      !“
```

```
Block 1
Block 2
Block 3
```

and information about where those blocks are:

TEXT

```
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
 B l o c k   2  !’   D N 2 ,   D N 4 ,   D N 5
 B l o c k   3  !’   D N 1 ,   D N 3 ,   D N 6
```

This is **metadata**.

The actual contents:

TEXT

```
 B l o c k   1  !’   a c t u a l   b y t e s
 B l o c k   2  !’   a c t u a l   b y t e s
 B l o c k   3  !’   a c t u a l   b y t e s
```

are on the DataNodes.

So remember:

TEXT

```
NameNode
      %
      % % %    m e t a d a t a

DataNodes
      %
      % % %    a c t u a l   b l o c k   d a t a
```

---

2. The NameNode has a filesystem image

The textbook describes the NameNode as maintaining an **image of the filesystem**.

Think of this as a saved representation of the filesystem's state.

For example, conceptually:

TEXT

```
Filesystem Image

/
% % %    u s e r s
%       % % %    a l i c e
%       %       % % %    f i l e A
%       % % %    b o b
%               % % %    f i l e B
% % %    d a t a
        % % %    f i l e C
```

The important thing is that this represents the **filesystem namespace and its metadata**, not the actual contents of the files.

So:

```
TEXT
Filesystem image
                !“
"What files/directories exist
 and what is their metadata?"
```

not:

```
TEXT
Filesystem image
                !“
"Here are the actual bytes of every file."
```

3. But there's a problem

Imagine the NameNode has this state:

```
TEXT
Filesystem:

/data
      % % %    f i l e A
      % % %    f i l e B
      % % %    f i l e C
```

Then users make thousands of changes:

```
TEXT
create fileD
delete fileB
create directoryX
rename fileA
...
```

If we had to completely rewrite the entire filesystem image after **every single operation**, that could be expensive.

So HDFS uses another mechanism: a **journal**.

4. The journal

Think of the journal as a record of **changes** made to the filesystem.

Suppose the current filesystem image says:

```
TEXT
/data
  % % %    f i l e A
  % % %    f i l e B
```

Then someone creates `fileC`.

Instead of immediately rewriting the entire filesystem image, the change can be recorded in the journal:

```text
Journal:

CREATE /data/fileC
```

Another operation:

```text
DELETE /data/fileB
```

becomes another journal entry:

```text
CREATE /data/fileC
DELETE /data/fileB
```

So conceptually:

```text
Current filesystem image
          +
     journal/change records
                  !“
Current filesystem state
```

Your textbook describes the NameNode as maintaining an image of the filesystem and a journal, and explains that after a restart the NameNode restores the checkpoint and applies the journal.

---

5. Why do we need both?

Think of the two as:

Filesystem image

A **snapshot of the filesystem state**.

```text
Image:
"Here is what the filesystem looked like at this point."
```

Journal

A **record of changes after that point**.

```text
Journal:
"After that snapshot, these things happened..."
```

For example:

```text
              Filesystem image
                                  %
```

```
                                %
                                %¼
             % % % % % % % % % % % % % % % % % % % % % % % % %
             %    / d a t a                                   %
             %     % % %    f i l e A                         %
             %     % % %    f i l e B                         %
             % % % % % % % % % % % % % % % % % % % % % % % % %
                                %
                                %    c h a n g e s
                                %¼
          Journal
                                %
                     % % % % % % % %4% % % % % % %
                     %                           %
      CREATE fileC   DELETE fileB
```

Together, they give the current state.

6. Now imagine the NameNode crashes

Suppose:

TEXT

```
NameNode
        Ø=Ü¥
```

The NameNode process stops.

But when it comes back, it needs to reconstruct its filesystem state.

It can use:

TEXT

```
Checkpoint / filesystem image
           +
        journal
```

Conceptually:

TEXT

```
Old checkpoint
      +
Journal entries
           !“
Reconstruct current filesystem state
```

For example:

TEXT

```
Checkpoint:

/data
  % % %    f i l e A
  % % %    f i l e B
```

Journal:

```
CREATE fileC
DELETE fileB
```

After applying the journal:

```
Current state:

/data
  % % %    f i l e A
  % % %    f i l e C
```

That is the basic reason the journal exists.

The textbook specifically states that when the NameNode restarts, it restores a checkpoint and applies the journal to reconstruct the current state.

---

7. But the journal can become huge

Imagine the NameNode has been running for months.

The journal could become:

```
CREATE A
DELETE B
CREATE C
RENAME D
CREATE E
DELETE F
...
millions of changes
...
```

Now restarting the NameNode could require processing a gigantic sequence of changes.

That's where **checkpointing** comes in.

---

8. Checkpoint

A checkpoint essentially creates a newer consolidated filesystem image.

Suppose we have:

```
Old checkpoint
      +
many journal entries
            !“
New checkpoint
```

Instead of keeping:

```text
Old image
+
10 million journal entries
```

we can periodically consolidate the state.

Conceptually:

```text
BEFORE

Checkpoint
     +
Journal
Journal
Journal
Journal
Journal
...
```

becomes:

```text
AFTER

New checkpoint
     +
newer journal
```

So the journal doesn't have to grow indefinitely from one ancient checkpoint.

The textbook describes the **Checkpoint Node** as periodically combining the checkpoint and journal.

---

9. Where does the Secondary NameNode come in?

This is where the name can be misleading.

You might naturally think:

> **"Secondary NameNode = backup NameNode."**

Do not memorize that.

For the material you're studying, its important role is related to **checkpointing**.

Conceptually:

```text
NameNode
        %
        %    f i l e s y s t e m   i m a g e   +   j o u r n a l
        %
        %¼
Secondary NameNode
        %
        %    p e r i o d i c a l l y   c r e a t e s   c h e c k p o i n t
```

```
       %¼
Checkpoint
```

The textbook says Secondary NameNodes can perform checkpointing or backup-related functions, and specifically describes the Checkpoint Node as periodically combining the checkpoint and journal.

So at this stage, remember:

> **Secondary NameNode is associated with checkpointing the NameNode's metadata state.**

Don't think:

```text
NameNode dies
      !“
Secondary NameNode automatically becomes NameNode
```

That is **not** the mental model you should use for the textbook's HDFS V1 discussion.

---

10. Why is this useful?

Suppose the NameNode has:

```text
Checkpoint:
State at time T1
```

and after that:

```text
Journal:
 c h a n g e s   f r o m   T 1  !’   T 2
```

If the NameNode restarts:

```text
Checkpoint
    +
Journal
        !“
State at T2
```

Without the journal, the NameNode would only know the filesystem as of T1.

Without the checkpoint, it could potentially have to process a very long history of changes.

Together:

```text
Checkpoint + Journal
                !“
Current filesystem state
```

---

11. One more detail from your textbook: protection against corruption

Your textbook also mentions that the journal/checkpoint information can be stored in **multiple directories** to guard against corruption or failure of one storage location.

Conceptually:

TEXT

```
                NameNode metadata
                                        %
                          % % % % % % % % %4% % % % % % % % %
                          !“                                  !“
          Directory 1     Directory 2
          checkpoint      checkpoint
          journal         journal
```

If one storage location has a problem, another copy provides protection.

Again, the overall principle is the same as replication elsewhere in HDFS:

> **Don't rely on one physical copy of something critical.**

Put the whole NameNode picture together

We can now expand our earlier architecture:

TEXT

```
                       HDFS
                                              %
                        % % % % % % % % % % % %4% % % % % % % % % % % % %
                        %                                                %
                        %¼                                               %¼
        NameNode                      DataNodes
        metadata                      actual data
                        %                                                %
            % % % % % % % %<% % % % % % %                                %
            %             %             %                                %
            %¼            %¼            %¼                               %¼
 Namespace  Image  Journal           Blocks
                                                                       %
                                   Replicas
```

And for NameNode recovery:

TEXT

```
        Checkpoint
             +
          Journal
                        %
                        %¼
      Current filesystem
           state
```

Checkpoint 6

Let's make sure this part is clear before moving to **Secondary NameNode + exact checkpoint process + HDFS file semantics**.

1. What is the difference between the **actual file data** and the **filesystem image**?

2. If the filesystem image represents the state at time T1, what is the purpose of the journal?

3. Why do we periodically create a checkpoint instead of allowing the journal to grow forever?

4. When a NameNode restarts, conceptually what two things does it use to reconstruct the current filesystem state?

5. Is the Secondary NameNode simply a hot replacement that automatically takes over if the NameNode crashes?

USER

Secondary NameNode is associated with checkpointing the NameNode's metadata state. checkpoint metadata state ? what does that mean ?

ASSISTANT

Yes— **“checkpointing the NameNode’s metadata state”** sounds more complicated than it is. Let's unpack exactly what **metadata state** means.

1. First: what is the NameNode's "state"?

The NameNode knows the **current structure of the HDFS filesystem**.

For example, suppose you have:

TEXT

```
/
% % %    u s e r s
%       % % %    a l i c e
%       %       % % %    r e p o r t . t x t
%       % % %    b o b
%               % % %    d a t a . c s v
```

The NameNode needs to remember things such as:

TEXT

```
What files exist?
What directories exist?
What are their names?
What blocks make up each file?
What metadata belongs to those files?
```

It also maintains the mapping between blocks and DataNodes.

For example:

TEXT

```
report.txt
        !“
Block 1
Block 2

 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 3
```

```
Block 2 !’ DN2, DN4, DN5
```

All of this information is metadata.

The actual contents:

```TEXT
Block 1 !’ "actual bytes of report.txt..."
```

are **not** the NameNode's metadata. Those bytes are on the DataNodes.

Your textbook describes the NameNode as maintaining the filesystem image and the mapping between file blocks and DataNodes.

---

2. So what does "metadata state" mean?

Simply:

> **Metadata state = what the NameNode currently knows about the filesystem.**

For example, at some point in time:

```TEXT
NameNode's state:

Files:
    /data/a.txt
    /data/b.txt

a.txt:
    Block 1
    Block 2

b.txt:
    Block 3

Block 1:
    DN1, DN2, DN3

Block 2:
    DN2, DN4, DN5

Block 3:
    DN1, DN3, DN6
```

That entire collection of information represents the **current metadata state**.

---

3. Then what is a checkpoint?

A **checkpoint** is essentially a saved, consolidated representation of that metadata state.

Imagine at 10:00:

```
NameNode's current state
                !“
       SAVE
                !“
   CHECKPOINT
```

The checkpoint says, conceptually:

> **"This is what the HDFS filesystem looked like at this point."**

It does **not** contain the actual contents of your files.

So:

```
CHECKPOINT
       %
       % % %   f i l e s y s t e m   s t r u c t u r e
       % % %   f i l e   m e t a d a t a
       % % %   b l o c k   i n f o r m a t i o n
       % % %   o t h e r   N a m e N o d e   m e t a d a t a
```

Not:

```
CHECKPOINT
       %
       % % %   a c t u a l   c o n t e n t s   o f   e v e r y   f i l e
```

---

4. Why do we need a checkpoint?

Because of the **journal** we just learned about.

Suppose our checkpoint is from 10:00:

```
CHECKPOINT at 10:00

/data
  % % %   A
  % % %   B
```

Then users make changes:

```
 1 0 : 0 5  !’  c r e a t e  C
 1 0 : 1 0  !’  d e l e t e  B
 1 0 : 1 5  !’  c r e a t e  D
 1 0 : 2 0  !’  r e n a m e  A
...
```

Those changes are recorded in the journal.

So we have:

TEXT

```
CHECKPOINT
     +
JOURNAL
          %
          % % %    c r e a t e   C
          % % %    d e l e t e   B
          % % %    c r e a t e   D
          % % %    r e n a m e   A
```

Together they represent the current state.

5. Checkpointing means "consolidate"

Eventually, instead of keeping:

TEXT

```
Old checkpoint
+
thousands/millions of journal entries
```

we create a **new checkpoint** representing the filesystem after those changes.

Conceptually:

TEXT

```
Old checkpoint
      +
Journal
            !“
NEW CHECKPOINT
```

Now we can start a new, smaller journal:

TEXT

```
NEW CHECKPOINT
      +
new journal
```

That's what **checkpointing** is accomplishing.

The textbook says the Checkpoint Node periodically combines the checkpoint and journal.

6. So why is the Secondary NameNode involved?

Now the sentence should make sense:

> **"Secondary NameNode is associated with checkpointing the NameNode's metadata state."**

It means:

```text
NameNode
      %
      %    h a s
      !“
Current filesystem metadata
      %
      %    p e r i o d i c a l l y   c h e c k p o i n t e d
      !“
Checkpoint
```

The Secondary NameNode participates in this checkpointing process rather than storing the actual user files.

VERY important:

```text
NameNode
        %
        % % %    m e t a d a t a

DataNodes
        %
        % % %    a c t u a l   f i l e   d a t a
```

The Secondary NameNode is concerned with the **NameNode's metadata/checkpoint information**, not with becoming a second DataNode or storing another copy of every user file.

One analogy

Imagine a college maintains a student database.

At 9 AM:

```text
Database snapshot:

 A l i c e  !’   e n r o l l e d
 B o b      !’   e n r o l l e d
 C h a r l i e  !’   e n r o l l e d
```

Then throughout the day:

```text
Bob drops course
David joins
Alice changes department
...
```

Instead of creating a completely new database snapshot after every change, you could keep:

```text
9 AM snapshot
+
list of changes
```

Periodically, you create a new snapshot incorporating all those changes.

That's roughly the idea behind:

```
TEXT
Checkpoint + Journal
                !“
New checkpoint
```

For HDFS:

```
TEXT
Filesystem metadata
                !“
Checkpoint
        +
Journal of changes
                !“
New checkpoint
```

So when I say **"checkpoint the metadata state,"** simply translate it in your head as:

> **"Make a saved snapshot of the current information that the NameNode uses to describe the HDFS filesystem."**

The next thing we should clarify is **exactly how the Secondary NameNode creates this checkpoint**, because that's the part where the roles of **NameNode, Secondary NameNode, filesystem image, and journal** all connect.

USER

Namespace Image Journal , what's the diff bw the three ?

ASSISTANT

They are related, but they mean **three different things** in the NameNode's metadata management.

First: Namespace

The **namespace** is the logical structure of the HDFS filesystem.

Think of it as:

```
TEXT
/
% % %    u s e r s /
%       % % %    a l i c e /
%       %       % % %    d a t a . c s v
%       % % %    b o b /
%               % % %    d a t a . c s v
% % %    l o g s /
        % % %    a p p . l o g
```

The NameNode maintains information such as:

```
TEXT
/data/file1.txt
        !“
```

```
Block 101 !’ DataNode 1, DataNode 2
Block 102 !’ DataNode 2, DataNode 3
```

So **namespace = what the filesystem currently looks like**.

It includes things like:

- files
- directories
- file !’ blocks
- block !’ DataNode mappings
- file metadata

It does **not** contain the actual contents of `file1.txt`. Those are on DataNodes.

Second: Image

The **image** is a **saved snapshot of the namespace/metadata state**.

Suppose at 10:00 the namespace is:

```text
/
%%%  data/
       %%%  A.txt
       %%%  B.txt
```

The NameNode can save that state as an **image**:

```text
IMAGE
------
data/
  A.txt
  B.txt
```

So:

> **Namespace = the current filesystem metadata in the NameNode's memory/state**

> **Image = a saved representation of that metadata state**

The textbook describes the NameNode as maintaining an image of the filesystem, together with a journal.

Third: Journal

The **journal** records **changes made after the image/checkpoint**.

Imagine the image was created at 10:00:

```text
IMAGE at 10:00
---------------
/data/A.txt
```

```
/data/B.txt
```

Then these things happen:

```
TEXT
 1 0 : 0 5  !’  c r e a t e  C . t x t
 1 0 : 0 7  !’  d e l e t e  B . t x t
 1 0 : 1 0  !’  c r e a t e  D . t x t
```

Instead of immediately creating a completely new image every time, the changes are recorded:

```
TEXT
JOURNAL
-------
CREATE /data/C.txt
DELETE /data/B.txt
CREATE /data/D.txt
```

Therefore:

```
TEXT
             IMAGE
               +
             JOURNAL
                              !“
       Current Namespace
```

This is the key relationship.

Put them side by side

Term	What is it?	Think of it as
Namespace	Current filesystem structure/metadata	Current state
Image	Saved snapshot of that metadata	Snapshot
Journal	Changes made after the snapshot	Change log

Simple analogy

Imagine a bank account.

At the beginning of the day:

```
TEXT
IMAGE:
 B a l a n c e  =  ¹ 1 0 , 0 0 0
```

Then transactions happen:

```
TEXT
JOURNAL:
 +  ¹ 2 , 0 0 0
 -  ¹ 5 0 0
```

```
 +   ¹ 1 , 0 0 0
```

Current state:

```
TEXT
 ¹ 1 0 , 0 0 0   +   ¹ 2 , 0 0 0   -   ¹ 5 0 0   +   ¹ 1 , 0 0 0
 =   ¹ 1 2 , 5 0 0
```

Similarly in HDFS:

```
TEXT
IMAGE
    !“
saved filesystem state

   +

JOURNAL
    !“
changes since that state

      !“

CURRENT NAMESPACE
```

Important: the journal does **not** contain the actual file data. It records changes to the **NameNode's metadata**. The actual file blocks remain on DataNodes.

And this is why **checkpointing** is useful: if the journal becomes very large, you can periodically create a new consolidated image instead of having to replay an enormous number of changes.

USER

ok, continue

ASSISTANT

Good. Now let's connect **Namespace + Image + Journal + Secondary NameNode** into one process, because that's the part that usually causes confusion.

1. What happens when the NameNode is running?

Suppose HDFS currently contains:

```
TEXT
/
% % %    d a t a /
        % % %    A . t x t
        % % %    B . t x t
```

The NameNode knows:

```text
A.txt !’ Block 1, Block 2
B.txt !’ Block 3
```

This information is the **namespace / metadata state**.

At some point, a saved copy of that state is created:

```text
IMAGE
----------------
/data/A.txt
/data/B.txt

A.txt !’ Block 1, Block 2
B.txt !’ Block 3
```

Now imagine the user creates `C.txt`.

The NameNode updates its current state:

```text
CURRENT NAMESPACE
-----------------
A.txt !’ Block 1, Block 2
B.txt !’ Block 3
C.txt !’ Block 4
```

But instead of immediately rewriting the entire image, the change is recorded in the **journal**:

```text
JOURNAL
-----------------
CREATE C.txt
C.txt !’ Block 4
```

So now:

```text
IMAGE
    !“
old saved state

+

JOURNAL
    !“
changes after image

=

CURRENT NAMESPACE
```

2. Why do we need the Secondary NameNode?

Imagine this happens for a long time:

```text
IMAGE
+
10 changes
+
100 changes
+
10,000 changes
+
1,000,000 changes
```

The journal keeps growing.

If the NameNode crashes and has to restart, it needs to reconstruct its state from:

```text
IMAGE + JOURNAL
```

A huge journal means more work.

So periodically, **checkpointing** happens.

The Secondary NameNode helps create a newer consolidated image from the existing metadata state and journal. The textbook describes this as periodically combining the checkpoint/image information with the journal.

Conceptually:

```text
OLD IMAGE
    +
JOURNAL
        !“
NEW CHECKPOINT / IMAGE
```

For example:

```text
Before:

IMAGE
A.txt
B.txt

JOURNAL
+ C.txt
+ D.txt
- B.txt
```

After checkpoint:

```text
NEW IMAGE
A.txt
C.txt
D.txt
```

Now the old changes have been incorporated into the new image.

3. Why not just keep the journal?

Because eventually you'd have:

```
TEXT
IMAGE from 6 months ago

+

millions of changes
```

The NameNode would have to replay all those changes to reconstruct the current state.

Checkpointing periodically gives you:

```
TEXT
Recent IMAGE
+
small JOURNAL
```

instead of:

```
TEXT
Very old IMAGE
+
huge JOURNAL
```

4. One VERY important distinction

Don't think:

> **Secondary NameNode = backup NameNode that automatically takes over.**

That's **not what the Secondary NameNode is in the Hadoop version described by your material**.

Its important role here is **checkpointing** the NameNode's metadata state.

Your textbook separately discusses the Backup NameNode and notes that in Hadoop V1, takeover was manual.

So remember:

```
TEXT
NameNode
      %
      % % %    m a n a g e s   c u r r e n t   n a m e s p a c e
      % % %    m a i n t a i n s   i m a g e
      % % %    m a i n t a i n s   j o u r n a l
                    %
                    !“
Secondary NameNode
      %
      % % %    h e l p s   c r e a t e   c h e c k p o i n t s
```

5. Now one more important HDFS topic: how DataNodes actually store blocks

So far we've said:

> **DataNodes store blocks.**

But **what does that actually mean inside a DataNode?**

The textbook says a DataNode stores each block using files in its local filesystem. A block is represented by:

```
TEXT
Block data
    +
Block metadata
```

The metadata includes things such as:

- **checksums**
- **generation stamp**

For example:

```
TEXT
DataNode 1

Block 101
% % %   a c t u a l  b l o c k  d a t a
% % %   m e t a d a t a
         % % %   c h e c k s u m
         % % %   g e n e r a t i o n  s t a m p

Block 102
% % %   a c t u a l  b l o c k  d a t a
% % %   m e t a d a t a
         % % %   c h e c k s u m
         % % %   g e n e r a t i o n  s t a m p
```

The **actual data** is the file content.

The **metadata** helps HDFS manage and verify that block.

Quick checkpoint before we continue

Suppose we have:

```
TEXT
IMAGE:
 A . t x t  !’  B l o c k  1 ,  B l o c k  2

JOURNAL:
create B.txt
 B . t x t  !’  B l o c k  3
```

What does the Secondary NameNode conceptually help produce from these two?

USER

ok , suppose if a user makes a file say a.txt , then it gets recorded in the journal , but the namespace is not modified , so if a user now tries to access the file a.txt , it might not be visible right ?

ASSISTANT

Exactly — **if the journal were the only thing recording the change and the current namespace were not updated, that would be a problem. But that's not how it works.**

The key thing I need to correct from the simplified explanation above is:

> **The journal is not a separate place where changes wait before becoming part of the namespace.**

When the user creates `a.txt`, the **NameNode's current metadata state is updated immediately**, and the change is also recorded in the journal for persistence/recovery.

Conceptually:

```
User: "Create a.txt"
                    %
                    %¼
     NameNode
                    %
                    % % % % % % % % % % % % % % % %º   U p d a t e   C U R R E N T   n a m e s p a c e
                    %                                  a . t x t   n o w   e x i s t s
                    %
                    % % % % % % % % % % % % % % % %º   R e c o r d   c h a n g e   i n   j o u r n a l
                           CREATE a.txt
```

So immediately afterward:

```
CURRENT NAMESPACE
-----------------
a.txt
```

and:

```
JOURNAL
-----------------
CREATE a.txt
```

Therefore, if the user immediately asks:

```
"Does a.txt exist?"
```

the NameNode already knows that **yes, it exists**.

Then what is the Image?

The image is just the **last saved checkpoint/snapshot**.

For example:

```
IMAGE
----------------
old state:
(no a.txt)
```

```
CURRENT NAMESPACE
----------------
a.txt exists


JOURNAL
----------------
CREATE a.txt
```

The image may be behind the current namespace. That's perfectly fine.

The relationship is:

TEXT

```
IMAGE
  +
changes recorded in JOURNAL
    !“
CURRENT NAMESPACE
```

So **the image does not have to be updated every time a file is created**.

That's precisely why the journal exists.

The textbook says the NameNode maintains an image of the filesystem and a journal, and on restart it restores the checkpoint and applies the journal to reconstruct the current state.

This distinction is very important:

- **Namespace** !’ what the NameNode currently knows
- **Image** !’ saved checkpoint of that knowledge
- **Journal** !’ persistent record of changes that happened after/around that checkpoint

So your concern was correct **if we had assumed "journal first, namespace later."** That assumption was the part to fix.

USER

ok , the image is stored as a snapshot in secondary namespace ? , journal is just for recording the changes to update the image later

ASSISTANT

Almost. **One correction:** the image is associated with the **NameNode's persistent metadata**, and the **Secondary NameNode helps create the checkpoint**. It isn't correct to think of it as "the image is stored inside the Secondary NameNode."

The clean model is:

TEXT

```
                NameNode
                                %
               % % % % % % % % % % % %4% % % % % % % % % % % %
               !“                                          !“
   CURRENT STATE             JOURNAL
```

```
   (namespace)               changes
                %                                                    %
                % % % % % % % % % % % %,% % % % % % % % % % % % % %
                                       %
              checkpointing
                                       !‘
           Secondary NameNode
                                       %
                                       !“
              NEW IMAGE
```

And your second statement is **basically correct**:

> **Journal records changes so that the saved image can later be brought up to date during checkpointing/recovery.**

For example:

TEXT

```
IMAGE:
A.txt
B.txt

CURRENT NAMESPACE:
A.txt
B.txt
C.txt

JOURNAL:
CREATE C.txt
```

During checkpointing:

TEXT

```
IMAGE + JOURNAL
            !“
NEW IMAGE:
A.txt
B.txt
C.txt
```

So remember these three in one sentence:

> **Namespace = current state; Image = saved checkpoint of that state; Journal = record of changes used to reconstruct/update the saved state.**

The textbook specifically describes the NameNode as maintaining the filesystem image and journal, with checkpointing periodically combining them.

USER

but what's the point in storing the image in the same namenode , the image is maintained to ensure that the file system is not corrupted , if im correct , then why ???

ASSISTANT

You're very close, but there is one important distinction:

> **The image is not primarily there to prevent filesystem corruption. It is there to provide a saved, recoverable representation of the NameNode's metadata state.**

And the fact that the image is stored on the **NameNode's persistent storage** is not pointless.

Think about what happens when the NameNode crashes

Suppose the NameNode currently knows:

```
TEXT
CURRENT NAMESPACE

 A . t x t  !’   B l o c k   1 ,   B l o c k   2
 B . t x t  !’   B l o c k   3
 C . t x t  !’   B l o c k   4
```

But this current state is in the NameNode's memory.

Now:

```
TEXT
Ø=Ü¥   N a m e N o d e   c r a s h e s
```

Its memory is gone.

So how does it reconstruct what the filesystem looked like?

It uses persistent metadata:

```
TEXT
IMAGE
+
JOURNAL
      !“
reconstruct namespace
```

The image gives it a **starting point**, and the journal gives it the changes that happened after that point. The textbook explicitly describes restart as restoring the checkpoint and then applying the journal.

But then why have a Secondary NameNode?

Because we don't want the NameNode to constantly rewrite its entire image.

Imagine:

```
TEXT
IMAGE
A
B
C
D
E
...
```

Then 1 million changes happen.

The journal becomes:

```
TEXT
+ file F
+ file G
- file B
+ file H
...
1,000,000 changes
```

Recovery would require:

```
TEXT
old IMAGE
    +
1,000,000 journal entries
```

That's inefficient.

So periodically:

```
TEXT
IMAGE + JOURNAL
              !“
NEW CHECKPOINT
```

The Secondary NameNode helps with this checkpointing process.

And here's the really important part

The **image and journal are persistent copies of metadata**.

The **DataNodes contain the actual file data**.

So imagine:

```
TEXT
                 HDFS
                                  %
                % % % % % % % % % %4% % % % % % % % % %
                !“                                      !“
    NameNode              DataNodes
                %                                       %
   metadata               actual data
                %
      % % % % % %4% % % % %
      !“                    !“
 Image     Journal
```

If the NameNode crashes:

```
TEXT
DataNodes still have:
A.txt's actual blocks
B.txt's actual blocks
C.txt's actual blocks
```

The NameNode needs to reconstruct:

TEXT

```
A . t x t  !’  B l o c k  1 ,  B l o c k  2
B . t x t  !’  B l o c k  3
C . t x t  !’  B l o c k  4
```

The image + journal help it recover that **metadata knowledge**.

So your statement should be changed to:

'L **"The image is maintained to ensure the filesystem isn't corrupted."**

Better:

' **"The image is a persistent checkpoint of the NameNode's metadata, so the NameNode can reconstruct its filesystem state after a restart. The journal records subsequent metadata changes, and checkpointing periodically creates a newer consolidated image."**

And **corruption detection** is a different mechanism in HDFS: DataNodes use **checksums** to detect corrupted block data.

That's an important distinction:

TEXT

```
NameNode Image + Journal
                !“
metadata persistence / recovery

DataNode Checksums
                !“
detect corrupted block data
```

USER

but if the namenode's memory crashed then the image and journals stored in it are also lost right ?

ASSISTANT

Yes — **if by “crashed” you mean the NameNode machine loses its persistent storage as well as its memory**, then the image and journal stored on that machine could be lost.

But normally, **a NameNode process/memory crash does NOT mean its disk data is lost**.

Think of two layers:

TEXT

```
NameNode machine
%
% % %    R A M
%         % % %    c u r r e n t   m e t a d a t a   i n   m e m o r y
%
% % %    D i s k
          % % %    I m a g e
          % % %    J o u r n a l
```

Case 1: NameNode process crashes

```text
 R A M  'L
 D i s k  '
```

The in-memory namespace disappears, but:

```text
Image + Journal
```

are still on persistent storage.

When the NameNode restarts:

```text
Image + Journal
              !“
reconstruct metadata
              !“
NameNode continues
```

This is exactly why the image and journal exist.

Case 2: NameNode machine's disk is destroyed

Now:

```text
 R A M  'L
 D i s k  'L
```

Then yes, the local image/journal could be lost.

This is why your textbook specifically mentions that journal/checkpoint information can be stored in multiple directories, providing protection against failure of the storage location.

So the idea isn't:

> **"Put the only copy of the image on the NameNode and we're safe."**

It's:

> **Persist the NameNode's metadata so a process/machine restart doesn't require reconstructing everything from scratch, while using additional storage arrangements to protect that persistent metadata.**

And notice another important distinction:

```text
NameNode failure
              !“
actual file blocks on DataNodes are still there
              !“
metadata must be recovered
```

The **DataNodes hold the actual data**, while the NameNode holds the information needed to understand and manage that data.

So your question is actually getting to the heart of **why HDFS separates metadata and data**.

USER

ok , if both image and journal are stored and modified in namenode , what exactly is the use of secondary node ?

ASSISTANT

The key is that the **Secondary NameNode is not mainly there to store a backup copy of the NameNode**.

Its main purpose in the Hadoop version you're studying is **checkpointing**.

Think of the responsibilities

The **NameNode** is doing the live work:

```
TEXT
NameNode
%
% % %   m a i n t a i n s   c u r r e n t   n a m e s p a c e
%
% % %   h a n d l e s   c l i e n t   r e q u e s t s
%
% % %   r e c o r d s   c h a n g e s   i n   t h e   j o u r n a l
%
% % %   m a i n t a i n s   t h e   c u r r e n t   m e t a d a t a   s t a t e
```

The **Secondary NameNode** periodically helps with the expensive maintenance operation:

```
TEXT
Secondary NameNode
%
% % %   c r e a t e s   a   c h e c k p o i n t
    by combining the existing image + journal
```

The textbook explicitly describes the Secondary NameNode as being used for checkpointing/backup-related purposes, and describes a Checkpoint Node as periodically combining the checkpoint and journal.

Why can't the NameNode just do this itself?

It **could**, conceptually.

But suppose the NameNode has:

```
TEXT
IMAGE
100 GB of metadata

+

JOURNAL
20 GB of changes
```

To create a new consolidated image, it has to process all of that metadata.

That's a potentially expensive operation.

Instead, the architecture separates the work:

```
TEXT
                  NameNode
                                    %
             normal operations
                                    %
                                    %
                                    %¼
             Journal grows
                                    %
                                    %
                    % % % % % % % % %4% % % % % % % % % %
                    %                                   %
                    %¼                                  %¼
     NameNode              Secondary
   keeps serving           NameNode
      clients              checkpoints
```

So the Secondary NameNode's value is essentially:

> **Take the accumulated metadata changes and periodically produce a fresh checkpoint, so the NameNode doesn't have to keep operating with an ever-growing journal.**

And here's the distinction that matters

Don't think:

```
TEXT
NameNode = primary
Secondary NameNode = backup
```

Instead, for **your textbook/Hadoop V1 material**, think:

```
TEXT
NameNode
      !“
LIVE filesystem management

Secondary NameNode
      !“
CHECKPOINTING of NameNode metadata
```

It is **not a hot standby that automatically takes over when the NameNode dies**. Your textbook separately discusses the Backup NameNode and notes that takeover in Hadoop V1 was manual.

One-line memory trick

NameNode = manages the filesystem. Secondary NameNode = periodically cleans up/consolidates its metadata history into a checkpoint.

USER

so , both are seperate servers , these namenode and secondary node , so this secondary node fetches the image and journal from the namenode and creates a new image and stores it in the namenode's persistent storage ?

ASSISTANT

Yes— **your mental model is now basically correct**, with one detail worth making precise.

In the Hadoop architecture you're studying, the **NameNode and Secondary NameNode are separate machines/processes**. The Secondary NameNode periodically works with the NameNode's metadata to create a checkpoint.

Conceptually:

TEXT

```
              NameNode
                    % % % % % % % % % % % % % % % % %
                    %    C u r r e n t              %
                    %    N a m e s p a c e          %
                    %                               %
                    %    I m a g e                  %
                    %    J o u r n a l              %
                    % % % % % % % % %,% % % % % % % %
                                     %
             metadata
             information
                                     %
                                     %¼
       Secondary NameNode
                    % % % % % % % % % % % % % % % % %
                    %    c o m b i n e s            %
                    %    I m a g e   +              %
                    %    J o u r n a l              %
                    % % % % % % % % %,% % % % % % % %
                                     %
                                     %¼
             New checkpoint
```

The important purpose is:

TEXT

```
Old Image + Journal
               !“
New consolidated Image
```

So instead of having:

TEXT

```
OLD IMAGE
+
10,000,000 journal changes
```

you eventually have:

TEXT

```
NEW IMAGE
+
```

```
only recent changes
```

One small correction

Don't think of the Secondary NameNode as simply:

> **"fetch image + journal !’ create image !’ copy image back to NameNode."**

The textbook's terminology is **checkpointing**, and it describes the Checkpoint/Secondary NameNode as periodically combining the checkpoint/image and journal.

The **exact mechanics of where the newly created checkpoint is stored/copied** aren't detailed in the textbook excerpt we're using, so I don't want to silently invent that part.

But the architectural idea you should remember for your course is:

> **NameNode manages the live filesystem; Secondary NameNode performs periodic checkpointing of the NameNode's metadata.**

And **Secondary NameNode "`automatic replacement/backup NameNode** in the Hadoop V1 material you're studying.

USER

ok , what are left ?

ASSISTANT

For **HDFS Architecture**, we've covered a lot. Based on your textbook section and the topics you said you want to learn, here's what remains.

' Already covered

1. **Why HDFS exists**
2. Cluster, node, distributed storage
3. **HDFS architecture**
4. **NameNode**
5. **DataNode**
6. **Blocks**
7. **Replication**
8. Namespace
9. Metadata vs actual data
10. **Read operation**
11. **Write operation / write pipeline**
12. Packets + acknowledgements
13. Checksums
14. Heartbeats
15. Block Reports

16. DataNode failure and under-replication
17. Rack awareness
18. **Image**
19. **Journal**
20. **Checkpoint**
21. **Secondary NameNode**
22. NameNode restart/recovery

Ø=Ý4 ill to cover

1. HDFS file semantics

This is important:

- Single writer
- Multiple readers
- Why HDFS uses this model
- Why files cannot be arbitrarily modified
- Append operation
- `hflush`

The textbook specifically discusses the **write-once / append** model.

---

2. DataNode's internal block storage

We mentioned this but haven't properly learned it:

```
TEXT

DataNode
%
% % %    B l o c k   d a t a
% % %    B l o c k   m e t a d a t a
           % % %    c h e c k s u m
           % % %    g e n e r a t i o n   s t a m p
```

We'll understand **what each of these actually means**.

---

3. Replication in more depth

We've understood the basic idea, but still need:

```
TEXT

Block placement
            !“
replica locations
            !“
under-replication
            !“
NameNode detects it
            !“
replication command
            !“
DataNode creates replica
```

And how HDFS avoids putting every replica in the same rack.

4. HDFS read/write optimization

Why HDFS is designed for:

- **high throughput**
- large files
- batch processing
- data locality
- minimizing network traffic

This is part of the reason HDFS doesn't behave like a normal desktop filesystem.

---

5. HDFS operations / functions

We'll connect the architecture to actual operations:

```
TEXT
Create
Read
Write
Append
Delete
Replication
Recovery
```

and understand **which component performs what**.

After that

We'll do one final **complete HDFS architecture walkthrough**:

```
TEXT
Client
    %
    %¼
NameNode
    %
    % % %    N a m e s p a c e
    % % %    I m a g e
    % % %    J o u r n a l
    %
    % % %    b l o c k   l o c a t i o n s
                   %
                   %¼
      DataNodes
            % % %    B l o c k
            % % %    B l o c k
            % % %    B l o c k
```

and trace:

"A user creates `A.txt` !' where does every piece of information go !' how is it replicated !' how is it read !' what happens if a DataNode dies !' what happens if the NameNode restarts?"

That will tie the whole HDFS architecture together.

USER

continue

ASSISTANT

Let's continue with the next topic: **HDFS file semantics**.

1. HDFS does not behave like a normal filesystem

In a normal filesystem, you might do:

```
Create file
Write to it
Change some bytes in the middle
Save it
```

HDFS is designed differently.

The textbook describes HDFS as having a **simple coherency model**:

> **one writer, multiple readers**

and files are generally **written once and can be appended to**, rather than arbitrarily modified.

So conceptually:

```
          HDFS file
                        %
              % % % % % % %4% % % % % %
               !“                      !“
   ONE WRITER   MANY READERS
```

Why?

Remember what HDFS is optimized for:

```
Huge files
        !“
Distributed across many machines
        !“
Replicated
        !“
High-throughput access
```

If many clients could simultaneously modify arbitrary portions of a distributed, replicated file, HDFS would need much more complicated synchronization.

Instead, it uses a simpler model.

2. Example

Suppose:

```
TEXT
logs.txt
```

A program is writing:

```
TEXT
Line 1
Line 2
Line 3
Line 4
```

Other programs can read the file.

But HDFS doesn't generally allow:

```
TEXT
Change Line 2
```

while treating the file like a normal random-access file.

Instead, you can **append**:

```
TEXT
Line 1
Line 2
Line 3
Line 4
 L i n e   5    !•   a p p e n d e d
 L i n e   6    !•   a p p e n d e d
```

So the important distinction is:

```
TEXT
MODIFY EXISTING CONTENT
               'L

APPEND NEW CONTENT
               '
```

The textbook specifically states that file contents cannot be updated, only appended.

3. Why "multiple readers" is okay

Suppose three programs want to read:

```
TEXT
A.txt
```

They can all read it:

TEXT

```
             A.txt
                              %
              % % % % % % % % %<% % % % % % % %
               !“              !“              !“
     Client  Client  Client
       1       2       3
```

There's no conflict because they're only **reading**.

The problem would be multiple writers simultaneously changing the same file.

Therefore:

TEXT

```
1 writer
+
many readers
```

keeps the consistency model much simpler.

---

4. What about appending?

HDFS does support appending to a file.

For example:

TEXT

```
Before:

A.txt
------
Hello
World
```

Append:

TEXT

```
!!!
```

After:

TEXT

```
A.txt
------
Hello
World
!!!
```

The important point is that we're **adding data at the end**, not changing:

TEXT

```
Hello
```

into:

```text
Goodbye
```

The textbook specifically discusses this write/append model and the `hflush` operation in its HDFS I/O section.

5. Now: what is `hflush`?

This is a term you should understand because it appears in the textbook.

Imagine a client is writing:

```text
A
B
C
D
```

The data is being sent through the HDFS write pipeline.

The client may want to say:

> **"Make sure the data I've written so far has been pushed through the pipeline so that readers can see it."**

That's where **`hflush`** comes in.

Conceptually:

```text
Client
    %
    %    w r i t e   A   B   C   D
    %¼
DataNodes
    %
    %    h f l u s h
    %¼
data written so far becomes visible
```

So don't think of `hflush` as:

> **"save the entire file permanently."**

It's about making the written data available/visible through the HDFS pipeline at that point.

The textbook discusses `hflush` in the context of HDFS's write pipeline and streaming I/O.

6. One important distinction

We now have several different things that sound like "writing":

```text
Client writes data
              !“
```

```
Data travels through DataNode pipeline
              !“
DataNodes store blocks
              !“
Checksums help detect corruption
              !“
hflush can make written data visible
```

Meanwhile, separately:

TEXT

```
NameNode
      !“
records filesystem metadata changes
      !“
Journal
      !“
Checkpoint/Image
```

So **file data** and **NameNode metadata** are two different streams of information.

File data

TEXT

```
C l i e n t  !’  D a t a N o d e s
```

Filesystem metadata

TEXT

```
C l i e n t  !’  N a m e N o d e  !’  m e t a d a t a  s t a t e / j o u r n a l
```

That separation is one of the central ideas of HDFS.

Quick check

If a client wants to change the **middle of an existing HDFS file**, rather than adding data at the end, is that part of the normal HDFS file model we've just learned? Why?

USER

you said hdfs only supports appending , but then its useless

ASSISTANT

It would be useless **if your goal were to use HDFS like a normal desktop filesystem** where you constantly edit small pieces of files.

But HDFS was designed for a different workload.

The key idea: HDFS is for huge datasets

Think about a dataset such as:

```
TEXT
10 TB of web logs
```

You typically don't want to do:

```
TEXT
change byte 4,283,917,221
```

Instead, the common pattern is:

```
TEXT
 D a y   1  !’   w r i t e   l o g s
 D a y   2  !’   a p p e n d   m o r e   l o g s
 D a y   3  !’   a p p e n d   m o r e   l o g s
...
```

So HDFS is very useful for:

- log storage
- large datasets
- data warehouses
- batch processing
- MapReduce/Spark processing
- storing files that are mostly written and then read many times

The textbook explicitly says HDFS is designed for **high-throughput access to large datasets**, with a simple coherency model and a write-once/append model.

Why avoid arbitrary modification?

Imagine a 1 TB file:

```
TEXT
1 TB file
%
% % %    B l o c k   1  !’   D a t a N o d e   A / B / C
% % %    B l o c k   2  !’   D a t a N o d e   D / E / F
% % %    B l o c k   3  !’   D a t a N o d e   G / H / I
% % %    . . .
```

Every block is replicated.

Now suppose you change something in the middle:

```
TEXT
Block 582
```

HDFS potentially has to deal with maintaining consistency among the replicas and coordinating distributed writes.

That's considerably more complicated than:

```
TEXT
write new data
           !“
```

```
create/extend blocks
            !“
replicate
```

The **one-writer/multiple-reader** model simplifies the consistency problem.

And "append only" doesn't mean "you can never change your data"

A common big-data pattern is:

TEXT

```
Old data
      !“
read/process
      !“
produce a new result
      !“
write a new HDFS file
```

For example:

TEXT

```
raw_logs/
    2026-10-01.log
    2026-10-02.log

                !“   S p a r k   p r o c e s s i n g

processed_logs/
    2026-10-01.parquet
    2026-10-02.parquet
```

You're not constantly editing the original massive file. You're **creating new datasets/results**.

So the important distinction is:

> **HDFS sacrifices convenient random modification in exchange for efficient, reliable, distributed storage and high-throughput processing of very large datasets.**

That's the design trade-off.

And for your syllabus, remember the textbook's exact conceptual model:

one writer + multiple readers + write once + append.

USER

ok , continue with what's left

ASSISTANT

The next part is **DataNode internal storage**, then we'll finish with **replication/recovery and the full HDFS flow**.

1. What exactly is stored inside a DataNode?

We already know:

> **DataNode stores blocks.**

But a block isn't just a single anonymous piece of data.

The textbook says a DataNode represents a block using **two files** in its local filesystem:

```
TEXT
DataNode
%
% % %    B l o c k   d a t a
%            % % %    a c t u a l   c o n t e n t s
%
% % %    B l o c k   m e t a d a t a
             % % %    c h e c k s u m s
             % % %    g e n e r a t i o n   s t a m p
```

Block data

This is the actual content.

Suppose:

```
TEXT
A.txt = "Hello Hadoop"
```

After HDFS splits the file into blocks, one of those blocks contains actual file bytes.

Those bytes are stored by the DataNode.

Block metadata

This is information **about the block**, rather than the contents themselves.

Two things mentioned in your textbook are:

Checksum

Used to help detect whether the block's data has become corrupted.

For example:

```
TEXT
Original block
            !“
checksum = X
```

Later:

```
TEXT
Read block
            !“
calculate checksum
            !“
compare with X
```

If they don't match, the data may have been corrupted.

Generation stamp

This is associated with identifying the current version/state of a block. The textbook mentions it as part of the block metadata.

2. How does the NameNode know which DataNode has which block?

This is where the **Block Report** becomes important.

Suppose:

```text
DataNode 1:
    Block 101
    Block 102
    Block 105
```

It periodically reports this to the NameNode:

```text
DataNode 1
          %
          %    B l o c k   R e p o r t
          %¼
NameNode

"I have:
  101
  102
  105"
```

The textbook says the Block Report contains information such as:

- block ID
- generation stamp
- length

The NameNode therefore maintains information about where replicas of blocks are located.

3. What happens when a replica disappears?

Suppose:

```text
Block 101

 D a t a N o d e   1  '
 D a t a N o d e   2  '
 D a t a N o d e   3  '
```

Replication factor = 3.

Now DataNode 2 fails:

```text
Block 101

 D a t a N o d e   1  '
 D a t a N o d e   2  'L
 D a t a N o d e   3  '
```

There are now only **2 replicas**.

That's called:

> **under-replication**

The NameNode detects this and identifies the block as needing replication.

The textbook describes the NameNode maintaining a priority queue for blocks that need replication, then instructing DataNodes to create replicas.

Conceptually:

```text
DataNode failure
           !“
replica count decreases
           !“
NameNode detects under-replication
           !“
NameNode chooses a DataNode
           !“
DataNode creates another replica
```

Eventually:

```text
Block 101

 D a t a N o d e   1  '
 D a t a N o d e   3  '
 D a t a N o d e   4  '
```

Back to 3 replicas.

---

4. What if there are TOO MANY replicas?

The opposite can also happen.

Suppose the desired replication factor is 3:

```text
Block 101

 D N 1  '
 D N 2  '
 D N 3  '
 D N 4  '
```

There are 4 copies.

That's **over-replication**.

The NameNode can instruct a DataNode to remove an unnecessary replica.

The textbook explicitly mentions that over-replicated blocks may have replicas removed.

So:

```
TEXT
Too few replicas
              !“
CREATE replica

Too many replicas
              !“
REMOVE replica
```

5. Why does rack awareness matter here?

We already covered the basic idea, but now connect it to replication.

Suppose:

```
TEXT
Rack A                Rack B

DN1                   DN4
DN2                   DN5
DN3                   DN6
```

If all three copies are:

```
TEXT
DN1
DN2
DN3
```

then a failure of **Rack A** could destroy all three copies.

Instead, HDFS tries to distribute replicas across racks.

For example:

```
TEXT
Block 101

 D N 1   ( R a c k   A )  '
 D N 2   ( R a c k   A )  '
 D N 4   ( R a c k   B )  '
```

Now:

```
TEXT
 R a c k   A   f a i l s  'L

 D N 1  'L
 D N 2  'L
```

```
D N 4  '
```

The data isn't completely lost.

At the same time, HDFS tries to avoid unnecessary network traffic and prefers nearby replicas when reading. The textbook describes this balance between reliability, availability, aggregate bandwidth, and network cost.

6. Data locality

Here's another important HDFS idea.

Suppose the client/application is running on:

```text
Machine A
```

and the block it needs is available on:

```text
Machine A
Machine B
Machine C
```

Instead of immediately reading across the network, HDFS prefers the **nearest replica**.

Conceptually:

```text
             Client
                          %
               % % % % % % % % %<% % % % % % % %
                !“              !“              !“
       DN-A    DN-B    DN-C
      nearby
                !‘
     prefer
```

Why?

Because network transfer costs time and bandwidth.

So HDFS tries to:

> **Move computation toward the data rather than unnecessarily moving huge amounts of data across the network.**

The textbook describes preferring the nearest DataNode for reads and tasks where the data resides.

7. So now we can connect almost everything

Suppose a user creates:

```
A.txt
```

Step 1 — NameNode

Client tells the NameNode:

```
"Create A.txt"
```

NameNode updates its filesystem metadata.

```
Namespace:
A.txt
```

The metadata change is recorded in the journal.

Step 2 — Blocks

Suppose A.txt is large enough to require:

```
Block 1
Block 2
Block 3
```

Step 3 — Placement

NameNode chooses DataNodes for replicas:

```
 B l o c k   1  !’   D N 1 ,   D N 2 ,   D N 4
 B l o c k   2  !’   D N 2 ,   D N 3 ,   D N 5
 B l o c k   3  !’   D N 1 ,   D N 3 ,   D N 6
```

Step 4 — Actual data transfer

The file contents **do not go through the NameNode**.

They go to the DataNodes:

```
Client
    %
    %¼
DataNode pipeline
    %
    % % %    r e p l i c a
    % % %    r e p l i c a
    % % %    r e p l i c a
```

The NameNode is dealing with **metadata and coordination**, not the actual file bytes.

This metadata/data separation is one of the central HDFS architecture principles in your textbook.

Step 5 — DataNodes report

DataNodes send:

```
TEXT
Heartbeat
```

to say:

> **"I'm alive."**

And:

```
TEXT
Block Report
```

to say:

> **"These are the blocks I currently have."**

Step 6 — Checkpointing

Meanwhile:

```
TEXT
NameNode
      %
      % % %    c u r r e n t   n a m e s p a c e
      % % %    i m a g e
      % % %    j o u r n a l
                        %
                        !“
      Secondary NameNode
                        %
                        !“
        checkpoint
```

Step 7 — Failure

If:

```
TEXT
 D N 2  Ø=Ü¥
```

the NameNode eventually determines that some blocks have fewer replicas than required.

Then:

TEXT

```
Under-replicated block
                !“
NameNode
                !“
instruct another DataNode
                !“
new replica
```

That's the complete HDFS architecture story.

The **only major thing left after this** is to study the **actual HDFS functions/operations and then do a final end-to-end architecture walkthrough + questions** so you can reproduce it in an exam/interview.

USER

what are hdfs functions ?

ASSISTANT

By **HDFS functions**, I mean the main things HDFS is responsible for doing as a distributed file system.

From the material we've been studying, we can group them into these functions:

1. File and directory management

HDFS provides the filesystem namespace:

TEXT

```
/
% % %    d a t a /
%       % % %    A . t x t
%       % % %    B . t x t
% % %    l o g s /
        % % %    a p p . l o g
```

The **NameNode** manages this namespace—creating, deleting, and maintaining information about files/directories.

---

2. File writing

When a client writes a file:

TEXT

```
Client
      %
      %¼
NameNode
      %
      %    d e c i d e s   b l o c k   p l a c e m e n t
      %¼
DataNodes
      %
      % % %    s t o r e   b l o c k s
```

The actual file data goes to DataNodes, not through the NameNode.

3. File reading

For reading:

```
TEXT
Client
      %
      %    " W h e r e   a r e   t h e   b l o c k s ? "
      %¼
NameNode
      %
      %    b l o c k   l o c a t i o n s
      %¼
Client
      %
      %¼
DataNodes
      %
      % % %    a c t u a l   d a t a
```

The NameNode provides the **locations**, while the client reads the actual data directly from DataNodes.

4. Block storage

HDFS breaks large files into **blocks** and stores those blocks across DataNodes.

```
TEXT
A.txt
    !“
Block 1
Block 2
Block 3

    !“   d i s t r i b u t e d

DN1   DN2   DN3 ...
```

DataNodes are responsible for storing and serving those blocks.

5. Replication and fault recovery

HDFS maintains multiple copies of blocks:

```
TEXT
Block 1
  % % %    D N 1
  % % %    D N 2
  % % %    D N 4
```

If a replica disappears, the NameNode detects under-replication and instructs DataNodes to create another copy. It can also remove excess replicas when over-replication occurs.

This is what gives HDFS **fault tolerance**.

6. Monitoring DataNodes

DataNodes continuously communicate with the NameNode.

Heartbeat:

> **"I'm alive."**

Block Report:

> **"Here are the blocks I currently have."**

The NameNode uses this information to track the health and block distribution of the cluster.

---

7. Data integrity

DataNodes maintain **checksums** for blocks.

Conceptually:

```
TEXT
Stored block
          !“
checksum
          !“
later verify
          !“
detect corruption
```

So HDFS can detect corrupted block data.

---

8. Efficient distributed access

HDFS also tries to make access efficient by:

- preferring nearby replicas when reading
- considering rack placement
- keeping replicas across racks for fault tolerance
- enabling computation close to where data resides

This reduces unnecessary network traffic while maintaining reliability.

So, in one picture

```
TEXT
                      HDFS
                                                %
                % % % % % % % % % % % % % % % % %<% % % % % % % % % % % % % % % % % % % %
                !“                              !“                                  !“
   Namespace          File I/O          Reliability
                %                               %                                   %
   NameNode         Read / Write       Replication
                %                               %                        F a u l t   r e c o v e r y
                %                               %                                   %
                % % % % % % % % % % % % %,% % % %4% % % % % % % % % % % % % % % % % % %
                                          !“
                  DataNodes
                                          %
               Store blocks
               Serve blocks
```

```
            Check integrity
            Report status
```

So when I said **"HDFS functions"**, I wasn't referring to some special list of mathematical functions. I meant **the operations/responsibilities that HDFS provides as a distributed file system**.

The next useful step is the **HDFS command/interface side**—how operations like `create`, `put`, `get`, `ls`, `cat`, `mkdir`, and `rm` map onto the architecture we've learned.

USER

so im doing a prepsentation , my topic is this , the rest are covered by others , is everything covered for this ?

ASSISTANT

Yes. Looking at the presentation scope in your sheet, **your assigned portion is essentially `i) HDFS architecture – functions`**, while the others are covering MapReduce, YARN, and Hive.

For **HDFS architecture + functions**, we have covered the core material you need.

What we've covered

Presentation requirement	Covered?
What is Hadoop/HDFS and why HDFS is needed	'
HDFS architecture	'
Cluster, nodes, racks	'
NameNode	'
DataNode	'
Secondary NameNode	'
Namespace	'
Image / checkpoint	'
Journal	'
Blocks	'
Block replication	'
Rack-aware replication	'
Heartbeat	'
Block Report	'
Checksums / data integrity	'
Read operation	'
Write operation	'

Presentation requirement	Covered?
Write pipeline	'
Acknowledgements	'
Data locality	'
DataNode failure	'
Under-replication	'
Over-replication	'
File semantics: one writer/multiple readers	'
Append	'
hflush	'
DataNode block storage	'
HDFS's major functions/responsibilities	'
NameNode restart/recovery	'
Checkpointing	'

These are all supported by the Elmasri HDFS material you've been using, particularly the HDFS architecture and I/O sections.

What I would not spend time on

Your sheet separately assigns:

- **MapReduce parallel programming model** !' other person
- **MapReduce sample program** !' other person
- **YARN scheduler** !' other person
- **Hive** !' other person

So you don't need to start learning those for **your HDFS presentation**.

One thing I would still do before your presentation

We have learned the concepts **individually**. Before making your slides, I'd do one final pass where we turn this into a **presentation-ready HDFS story**:

TEXT

```
What is HDFS?
            !“
Why do we need it?
            !“
HDFS Architecture
            !“
 N a m e N o d e  % % % % % % % % %    D a t a N o d e s
      %                                          %
      %                                          % % %    B l o c k s
      %                                          % % %    R e p l i c a s
```

```
      %                                   % % %    C h e c k s u m s
      %
      % % %    N a m e s p a c e
      % % %    I m a g e
      % % %    J o u r n a l
      % % %    B l o c k   l o c a t i o n s
            !“
Read flow
            !“
Write flow
            !“
Replication & fault tolerance
            !“
Rack awareness
            !“
Secondary NameNode / checkpointing
            !“
HDFS functions
```

Then we can turn that directly into **what you should put on each presentation slide**, including the diagrams you should draw and what you should actually say while presenting.

That would be the final step for your assigned topic.

If you want, I can:

- Explain HDFS write pipeline details
- Describe HDFS read operations
- Discuss HDFS failure recovery process